Presentation Project : jmeter

ANEES UR REHMAN

22P-9115

SE 5A

What is JMeter and Its Uses?

Apache JMeter, an open-source performance testing tool, is widely recognized for its capability to test web applications, databases, and services under various conditions. This report explores JMeter's core components, types of performance tests it supports, and how to effectively use the tool to simulate real-world traffic loads and ensure robust performance.

It Supports protocols like **HTTP, FTP, SOAP, REST, JDBC**, and more.

.**Types of Performance Testing**

JMeter supports several types of performance testing, enabling testing of web applications, services, and APIs under various conditions:

Load Testing: This is the most common form of performance testing, designed to assess the behavior of a system under normal and peak load conditions. The goal is to identify the maximum number of users an application can handle without performance degradation.

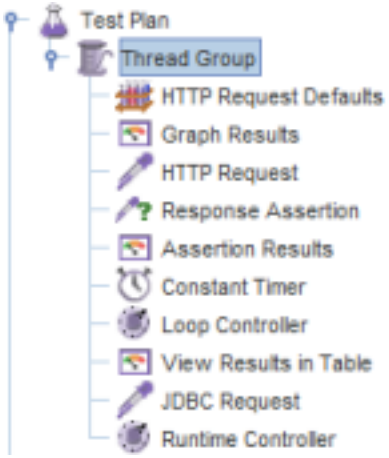
Stress Testing: Stress testing involves subjecting the system to extreme load conditions to determine its breaking point. This helps in understanding how the system behaves when pushed beyond its limits and whether it fails gracefully.

API Testing: JMeter can be used to test APIs by sending requests and verifying the accuracy and performance of the responses returned by the server.

Security Testing: While not its primary function, JMeter can perform basic security tests, such as simulating heavy traffic to check how the application handles potential vulnerabilities, or testing the system’s response to various attack scenarios.

What is a Test Plan?

Test Plan is where you add elements required for your JMeter Test.
It stores all the elements (like ThreadGroup, Timers etc) and their corresponding settings



required to run your desired Tests.

1. Thread Group

What is a Thread?

A **thread** is a basic unit of execution in a program. In JMeter, threads simulate multiple virtual users who send requests to the server simultaneously.

Purpose of Threads

Threads allow you to mimic real-world usage scenarios by executing tasks concurrently. This is particularly useful in load testing and performance testing of web applications or services.

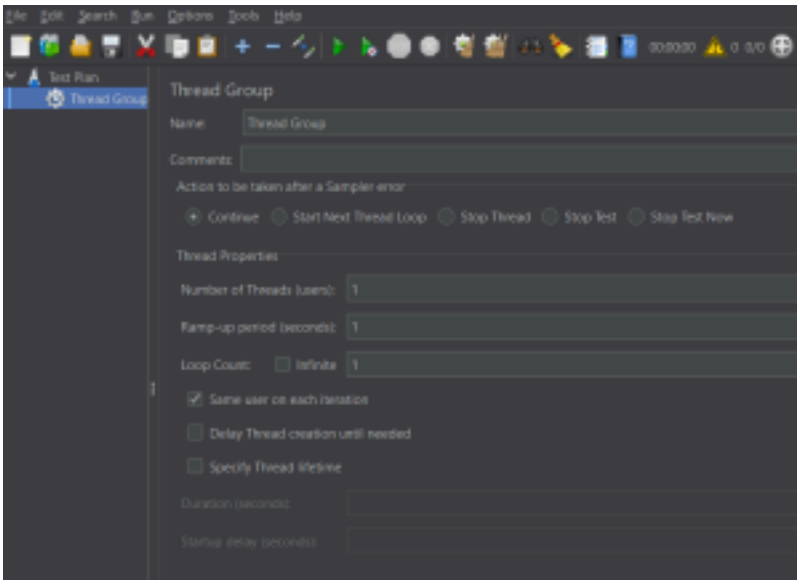
What is a Thread Group?

A **Thread Group** is the starting point of a JMeter test plan. It defines:

- The number of threads (users).
- The ramp-up period (time taken to start all threads).
- The number of iterations (how many times the test will run).

Purpose of a Thread Group

The Thread Group serves as a container for threads and other elements like samplers, listeners, and configuration components. It controls the overall execution flow of the test.



2. Samplers

What are Samplers?

Samplers in JMeter are elements that send requests to a server and wait for a response. These requests can be of various types, such as HTTP, FTP, JDBC, etc. Each type of request is represented by a specific sampler. Samplers allow you to test different types of communication protocols and services, helping you simulate real-world user interactions and measure the server's response.

Types of Samplers

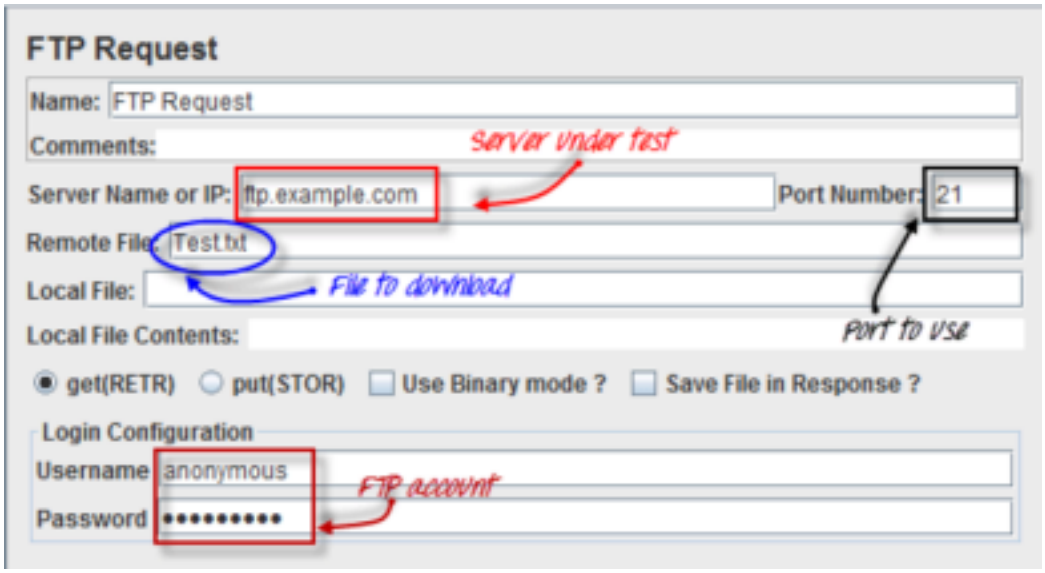
2.1 FTP Sampler

What is FTP?

FTP (File Transfer Protocol) is a standard network protocol used for transferring files between a client and a server over a TCP-based network. It is commonly used to upload and download files to/from a server.

FTP Sampler

Definition: The **FTP Sampler** in JMeter is used to simulate FTP requests, such as uploading or downloading files from an FTP server. **Purpose**: This sampler helps you measure the performance of FTP servers by simulating multiple file transfer requests under load conditions. It allows you to test scenarios like file uploads, downloads, and directory listings.
For example, if you want to download a file “Test.txt” from an FTP server under test, you need to configure some parameters in JMeter as the figure below JMeter will send FTP command to FTP server ftp.example.com and then download a file Test.txt from that server.

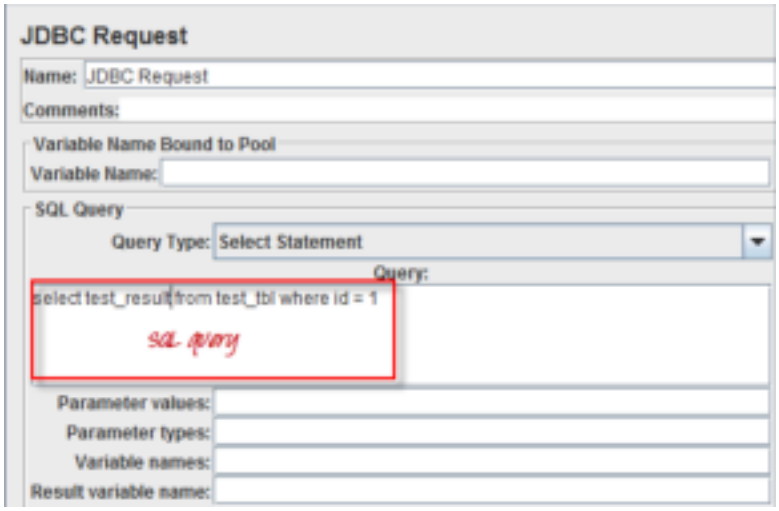


2.2 JDBC Request

What is JDBC?

JDBC (Java Database Connectivity) is an API that allows Java applications to interact with databases. It provides methods for querying and updating data in relational databases, such as MySQL, Oracle, and PostgreSQL.**JDBC Sampler**

Definition: The **JDBC Request Sampler** is used to send SQL queries (such as SELECT, INSERT, UPDATE, DELETE) to a database and retrieve the results. **Purpose**: This sampler allows you to test and measure the performance of database interactions. It is used to validate database operations, check the response times for queries, and measure the impact of database load during stress tests. For example, a database server has a field test_result stored in a table name test_tbl. You want to query this data from the database server; you can configure JMeter to send a SQLquery to this server to retrieve data.



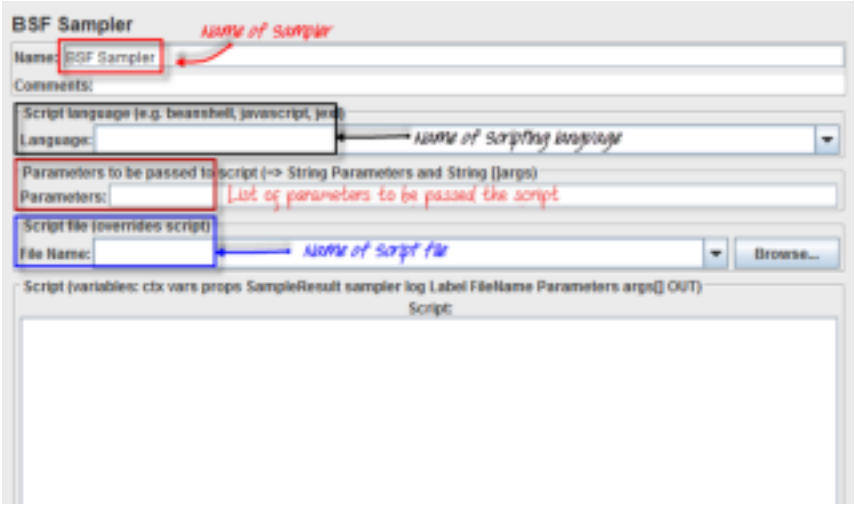
2.3 BSF Sampler

What is BSF?

BSF (Bean Scripting Framework) is a framework that enables the integration of different scripting languages (such as JavaScript, Groovy, Jython, etc.) with Java applications. It allows users to execute scripts within Java programs, including JMeter.

BSF Sampler

Definition: The **BSF Sampler** in JMeter allows you to write scripts in different languages (e.g., Groovy, JavaScript) and execute them as part of your test plan. **Purpose**: This sampler is useful when you need to perform custom scripting or logic as part of your performance tests. It allows for flexible, dynamic execution based on scripts, making it an advanced tool for complex testing scenarios.



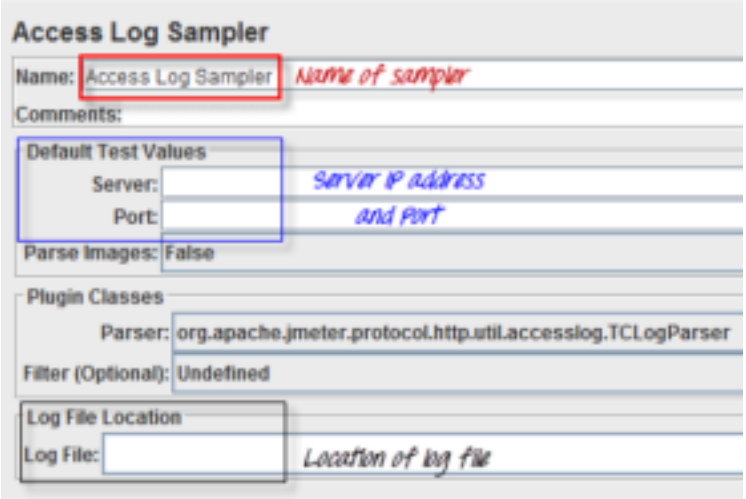
2.4 Access Log Sampler

What is an Access Log?

An **Access Log** is a file generated by web servers that logs requests made to the server, including details like the requesting IP address, requested URLs, response status codes, and timestamps. These logs are crucial for analyzing web traffic and diagnosing performance issues.

Access Log Sampler

Definition: The **Access Log Sampler** simulates requests based on real access logs from a server. It can replay a set of historical requests by using a log file, providing a more realistic testing scenario. **Purpose**: This sampler is used for load testing by reproducing real-world traffic patterns from access logs. It helps you simulate actual user requests and understand how the system performs under real-world conditions.



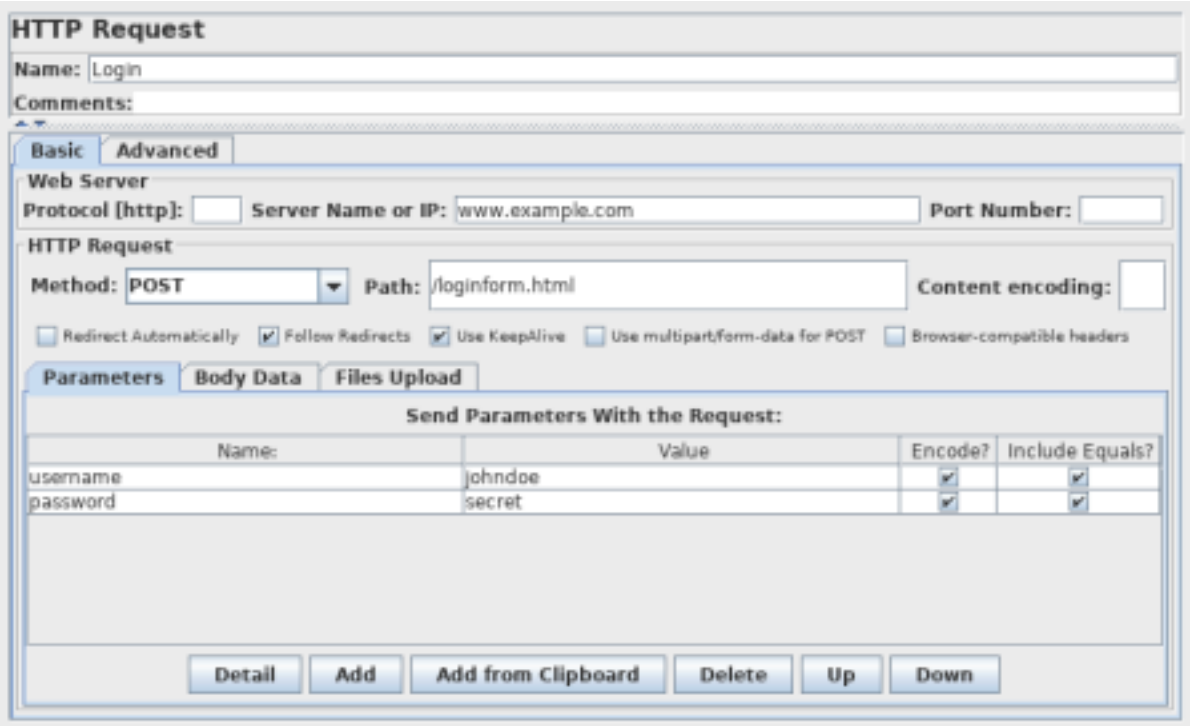
2.5 HTTP Request Sampler

What is HTTP?

HTTP (Hypertext Transfer Protocol) is the foundation of data communication on the web. It is the protocol used for transmitting data between a web browser (client) and a web server.

HTTP Sampler

Definition: The **HTTP Request Sampler** sends HTTP or HTTPS requests to a web server to simulate real user interactions, such as opening web pages or submitting forms. **Purpose**: The HTTP Sampler is used to measure the performance of web applications. It helps test how well the server responds to different HTTP requests, such as GET, POST, PUT, and DELETE, under load.



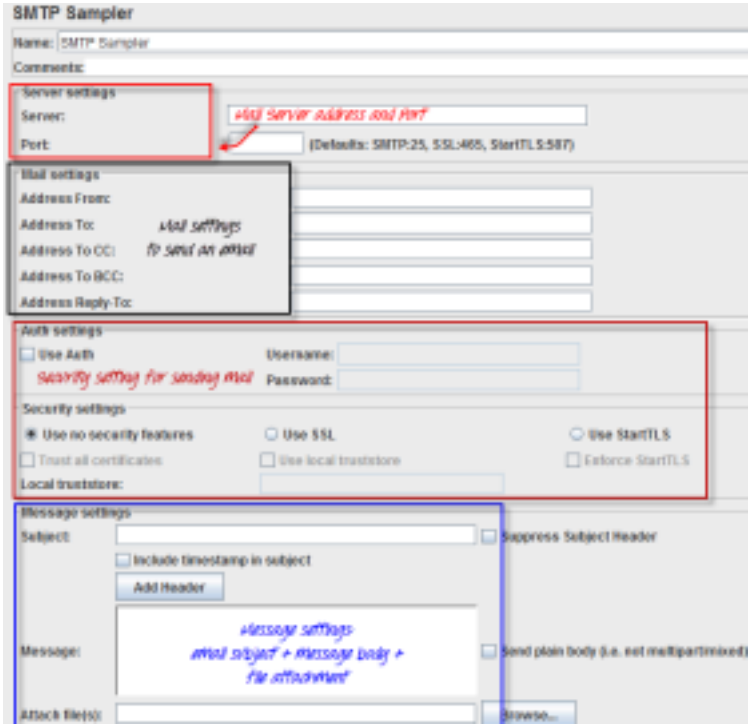
2.9 SMTP Sampler

What is SMTP?

SMTP (Simple Mail Transfer Protocol) is a protocol used for sending email messages between servers. It is the standard protocol for email transmission.

SMTP Sampler

Definition: The **SMTP Sampler** in JMeter is used to simulate sending email messages via an SMTP server. **Purpose**: This sampler is useful for testing the performance of email servers and for simulating email-related operations, such as sending automated emails in an application or load-testing an email service.



3. Listeners

What are Listeners?

Listeners in JMeter are elements that capture and display the results of a test execution. They record the request/response details and present this data in various formats such as tables, graphs, and logs. These visualizations help analyze the test results and identify performance issues.

Types of Listeners and Their Purposes

3.1 View Results Tree

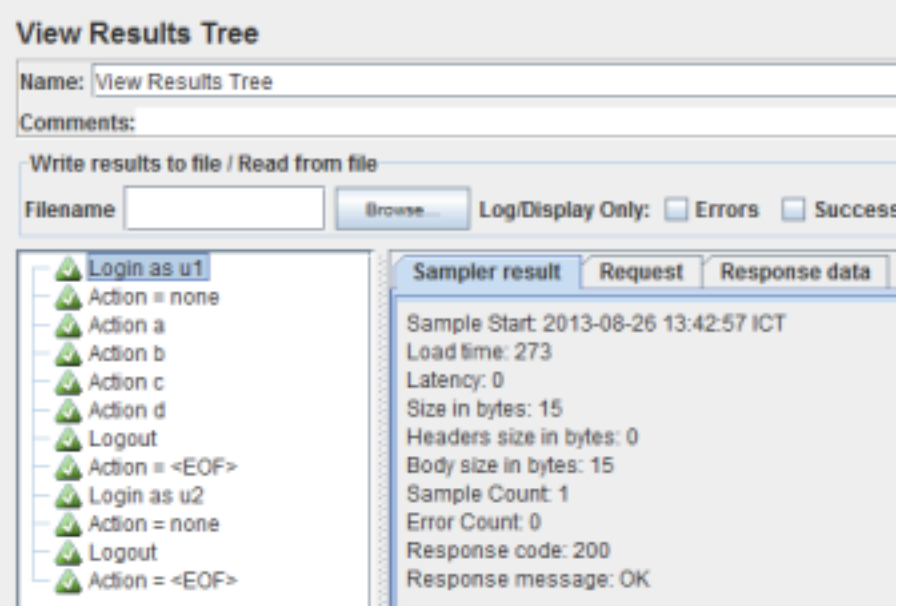
What is the View Results Tree?

The **View Results Tree** listener shows the results of each request in a detailed tree view, including request and response information.

Purpose:

The **View Results Tree** is primarily used for debugging. It provides detailed information for each request, such as request headers, parameters, and responses, helping testers understand the

interaction between client and server. It's most useful during test development to diagnose issues



3.2 Graph Results

What is the Graph Results?

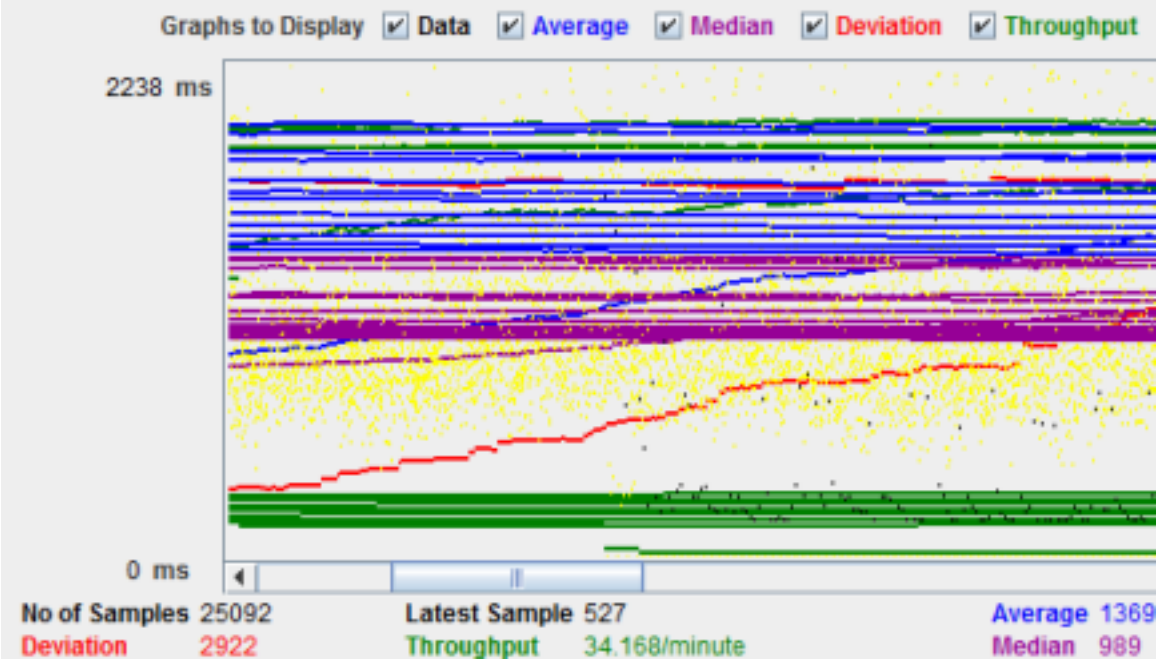
The **Graph Results** listener provides a graphical representation of the test results, displaying trends over time.

Purpose:

This listener helps visualize performance data, such as:

- **Response Time vs. Time**,
- **Throughput vs. Time**,
- **Response Times per Request**.

It’s useful for quickly identifying performance issues and patterns, such as spikes in response time or throughput drops.



3.3 View Results in Table

What is the View Results in Table?

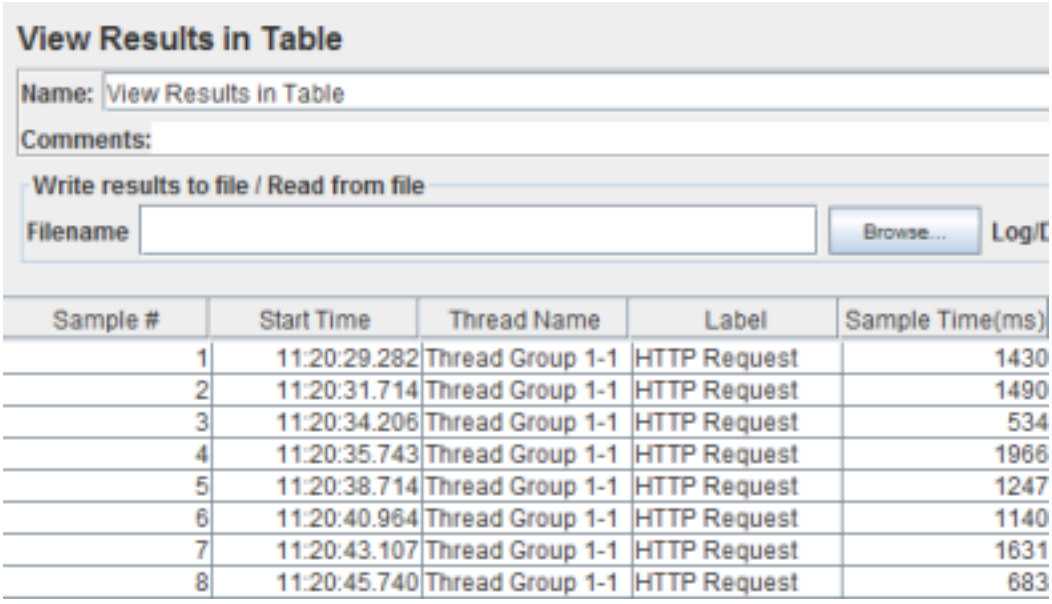
The **View Results in Table** listener presents each request's results in a tabular format.

Purpose:

The **View Results in Table** listener is designed to show detailed information for each request, including:

- Request URL,
- Response time,
- Status code (success or failure),
- Response data.

It's ideal for inspecting individual requests and their results and is also useful for exporting data for further analysis.

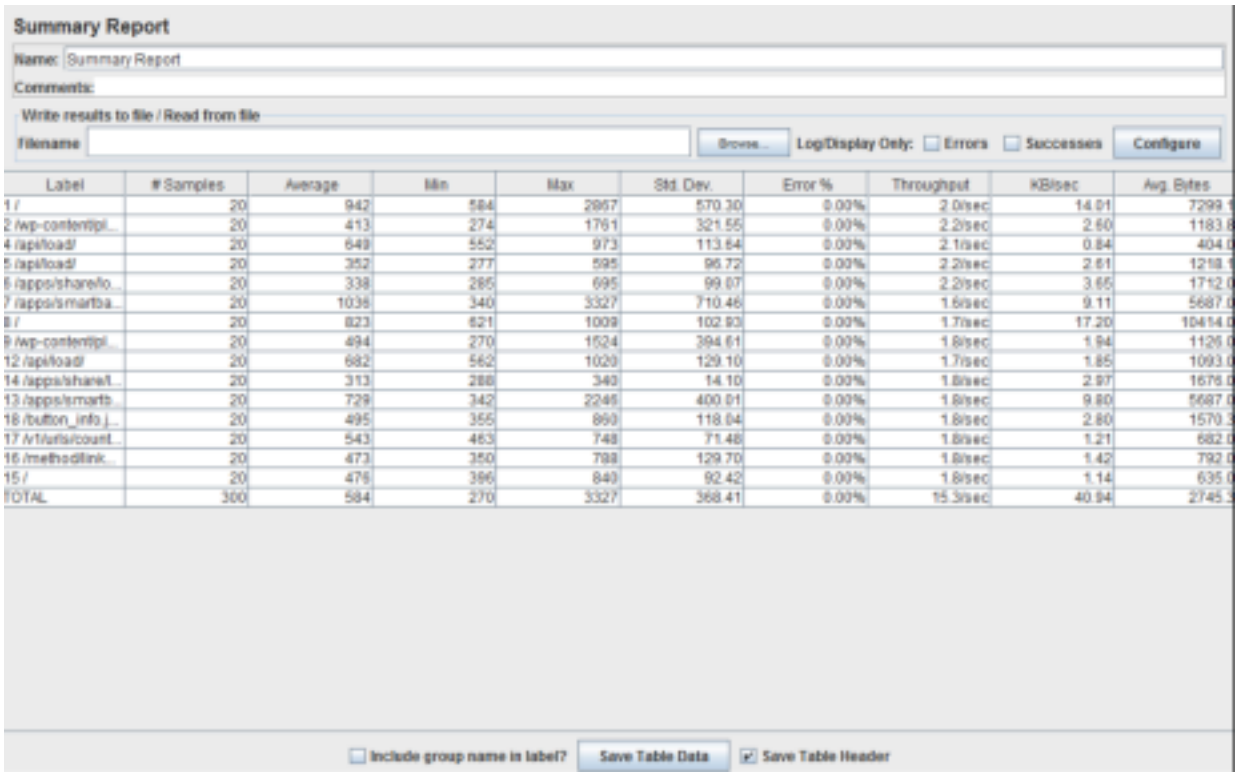


Sample #	Start Time	Thread Name	Label	Sample Time(ms)
1	11:20:29.282	Thread Group 1-1	HTTP Request	1430
2	11:20:31.714	Thread Group 1-1	HTTP Request	1490
3	11:20:34.206	Thread Group 1-1	HTTP Request	534
4	11:20:35.743	Thread Group 1-1	HTTP Request	1966
5	11:20:38.714	Thread Group 1-1	HTTP Request	1247
6	11:20:40.964	Thread Group 1-1	HTTP Request	1140
7	11:20:43.107	Thread Group 1-1	HTTP Request	1631
8	11:20:45.740	Thread Group 1-1	HTTP Request	683

3.4 Summary Report

What is the Summary Report?

The **Summary Report** listener presents test results in a concise, tabular format, displaying key performance metrics.



Purpose:

The **Summary Report** aggregates results across all requests, showing metrics such as:

- **Throughput** (requests per second),
- **Average Response Time**,
- **Error Percentage**,
- **Min/Max Response Time**.

This listener is commonly used to get a quick overview of the system's performance during load or stress tests.

3.5 Aggregate Report

What is the Aggregate Report?

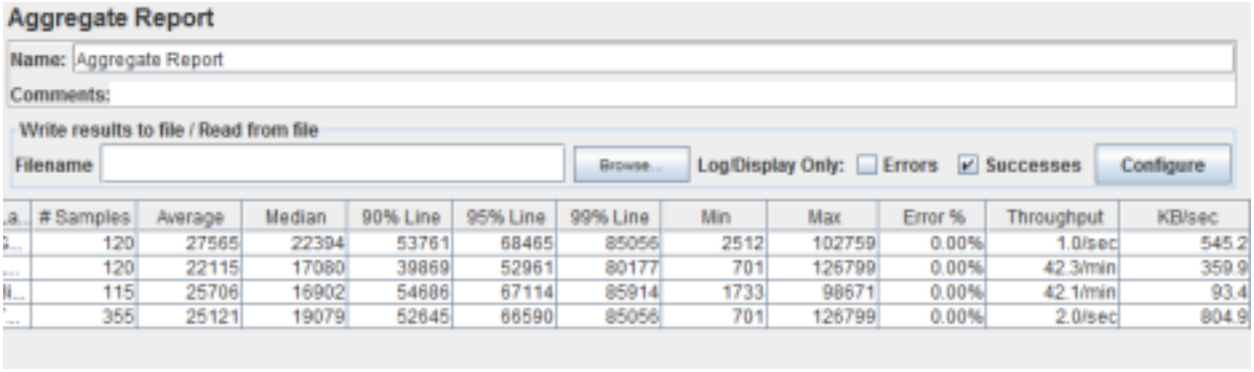
The **Aggregate Report** listener displays a table of aggregated statistics, similar to the **Summary Report**, but it groups results based on the type of request.

Purpose:

The **Aggregate Report** is useful for tracking the cumulative performance of different requests over the entire test duration. It includes metrics like:

- **Total Samples**,
- **Average Response Time**,
- **Error Count**,
- **Throughput**.

This listener is helpful when analyzing large test results with multiple request types and examining their overall performance.



a...	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Max	Error %	Throughput	KB/sec
...	120	27565	22394	53761	68465	85056	2512	102759	0.00%	1.0/sec	545.2
...	120	22115	17080	39869	52961	80177	701	126799	0.00%	42.3/min	359.9
ll...	115	25706	16902	54686	67114	85914	1733	98671	0.00%	42.1/min	93.4
...	355	25121	19079	52645	66590	85056	701	126799	0.00%	2.0/sec	804.9

4. Configuration Elements

What are Configuration Elements?

Configuration Elements in JMeter are used to configure parameters that are common to multiple samplers or other elements. They allow you to set default values or manage variables

that will be used throughout the test plan. These elements help avoid repetition, reduce redundancy, and ensure consistency across different parts of the test.

4.1 HTTP Request Defaults

What is the HTTP Request Defaults?

The **HTTP Request Defaults** element is used to specify default values for HTTP request samplers. It provides a way to centralize and manage HTTP request configurations like the server name, port, and other HTTP-specific parameters.

Purpose:

The **HTTP Request Defaults** element is helpful when multiple HTTP requests are made to the same server or domain. By setting common parameters, it saves time and ensures consistency. Common configurations include:

- **Server Name/IP**: The address of the target server.
- **Port**: The port number to use for the HTTP requests (e.g., 80 for HTTP or 443 for HTTPS).
- **Protocol**: HTTP or HTTPS.

This element reduces the need to repeat these parameters for every HTTP request in the test plan.

4.2 CSV Data Set Config

What is the CSV Data Set Config?

The **CSV Data Set Config** element allows you to import data from a CSV (Comma-Separated Values) file and use it as input for parameterized testing. This is particularly useful when you want to test the system with different sets of input data.
Purpose:

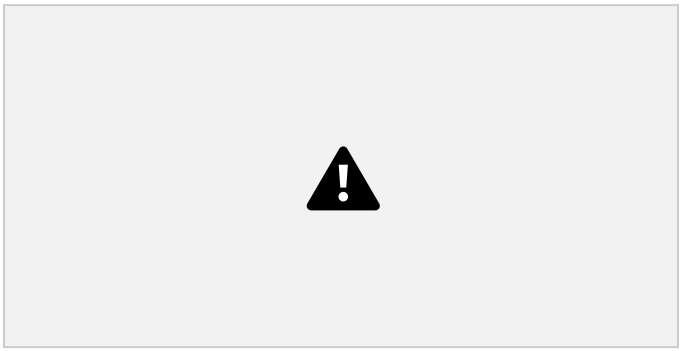
The **CSV Data Set Config** element is used to parameterize the test by reading values from a CSV file. Each row of the CSV file represents a new set of values for the test. Common use cases include:

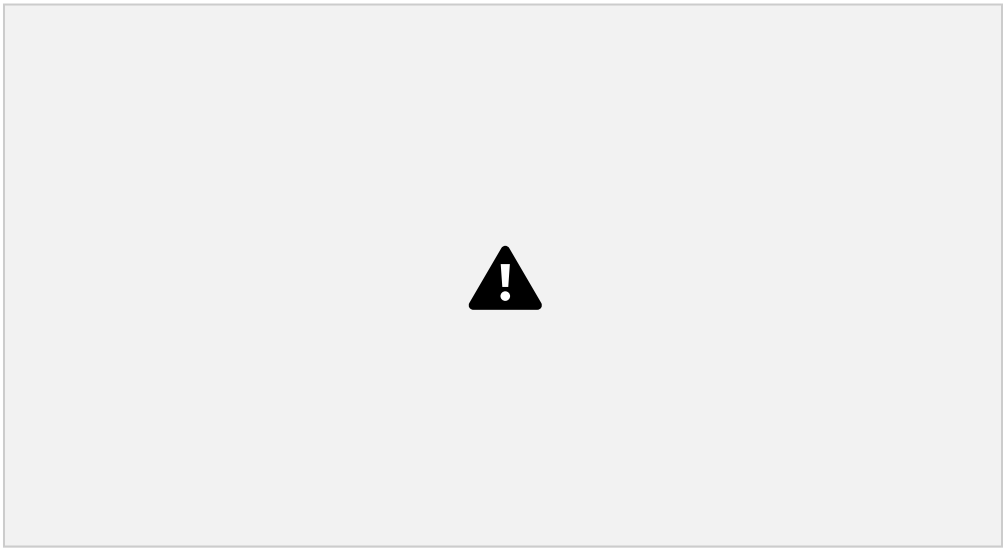
- Passing different login credentials in each iteration (e.g., username and password),

The element supports features such as:

- Defining variable names that match the columns in the CSV file,
- Handling file read/write operations,

- Iterating through the file in each thread.





4.3 HTTP Cookie Manager

What is the HTTP Cookie Manager?

The **HTTP Cookie Manager** is used to manage cookies in JMeter. Cookies are small pieces of data stored on the client-side (browser) that are sent with HTTP requests to the server.
Purpose:

The **HTTP Cookie Manager** automatically handles cookies during the test. It captures cookies returned from the server and sends them with subsequent requests to simulate the behavior of a real web browser. This is useful for:

- Maintaining session state across multiple requests,
- Handling authentication or user-specific data stored in cookies.

The element also supports the ability to configure cookies explicitly if needed, including expiration times, domains, and paths.

HTTP Proxy Server in JMeter: Record Example Script

What is a Proxy?

A **Proxy Server** is an intermediary server that sits between the user's browser (or client) and the target web server. It intercepts the communication between the two, allowing you to capture and analyze requests and responses.

In the Context of JMeter:

JMeter's **Proxy Server** enables you to record HTTP/HTTPS traffic between your browser and the web server. This helps you capture real user actions in your browser, which can be converted into test scripts. Recording with a proxy server is a quick and efficient way to create test plans without having to manually configure each request.

When the **JMeter Proxy Server** is enabled, it acts as a man-in-the-middle, allowing JMeter to record the actions you perform in your browser. This helps in automating the creation of test scripts that simulate real user behavior. You can then replay these recorded actions under different conditions to assess the performance and scalability of your web applications.

What is Recording Test Scripts?

Recording Test Scripts is the process of capturing user interactions with a web application, so that these interactions can be used as part of a performance test plan in JMeter.

Purpose of Recording Test Scripts:

The purpose of recording test scripts is to generate test scenarios that mimic real user activity on a web application. This process involves recording HTTP or HTTPS requests made by a user while interacting with the application, and saving them as JMeter requests (called **samplers**).

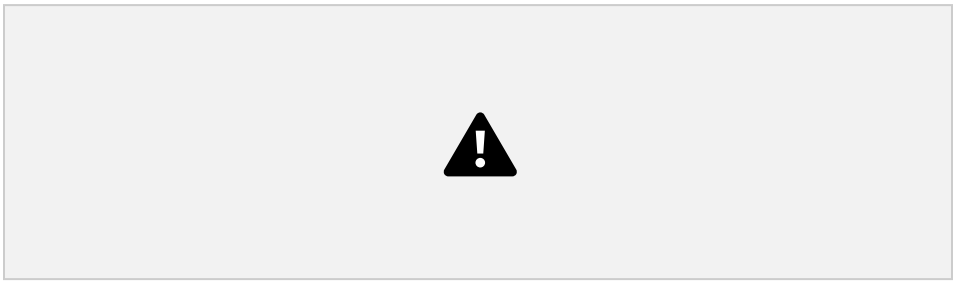
Recording test scripts helps to:

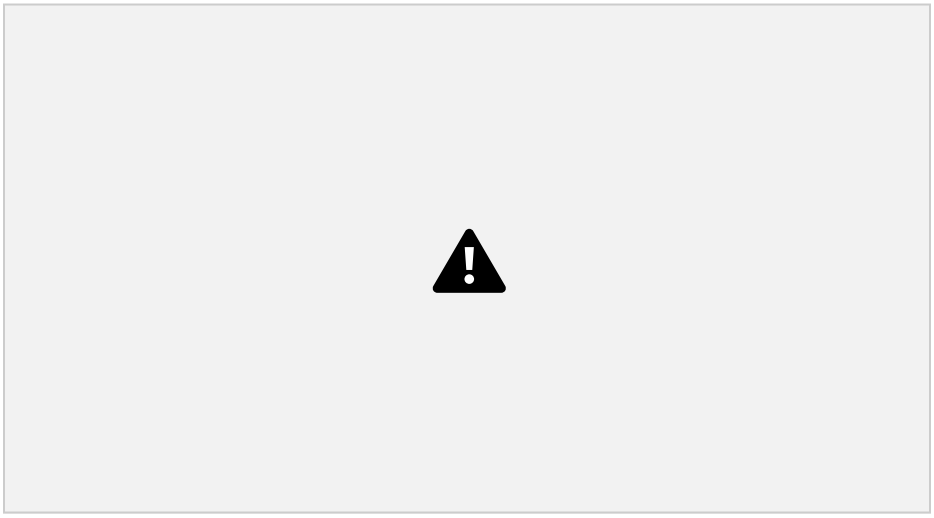
- **Automate test creation**: Instead of manually creating each HTTP request, the **Proxy Server** automatically generates the requests based on your interactions. ● **Mimic real user behavior**: By recording actual user actions (e.g., logging in, submitting forms), JMeter test scripts reflect the behavior of real users.

- **Identify performance bottlenecks**: By recording how users interact with the system, you can replicate the same flow and then run tests to measure system performance under load.

Step 1 : Setting up the HTTP proxy server

Firstly create a thread group under the test plan.

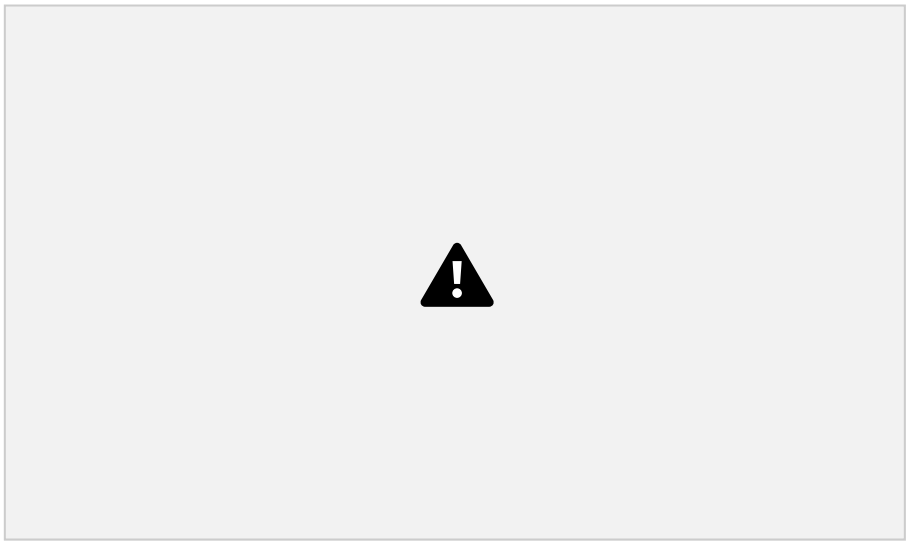




Add **HTTP Request**

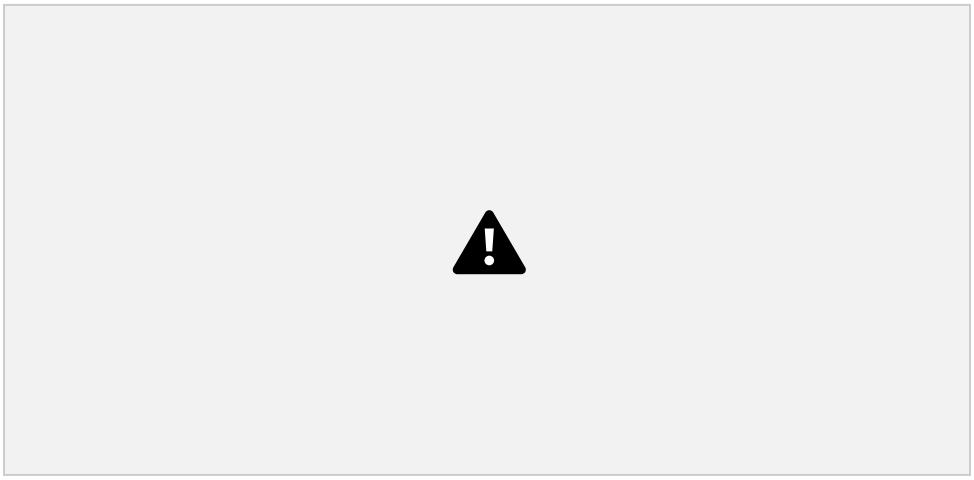
Select the Thread Group; right click **Add** => **Config Element** => **HTTP Request**

Defaults

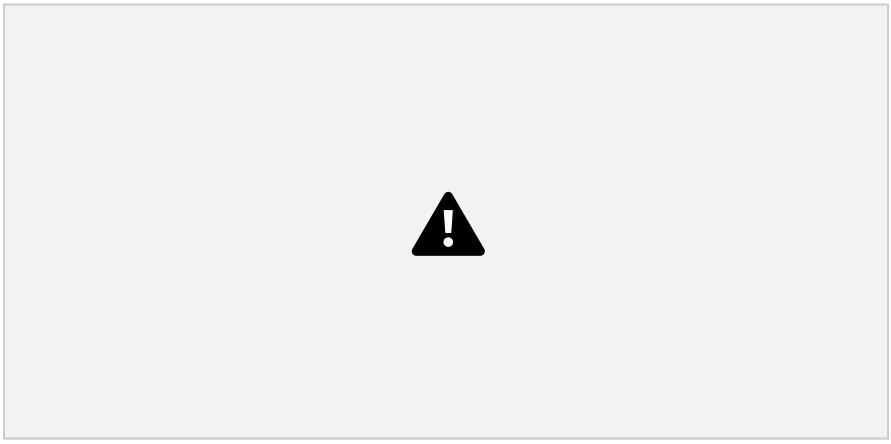


In HTTP Request Defaults element: In Server name or IP, enter “google.com”. You should keep

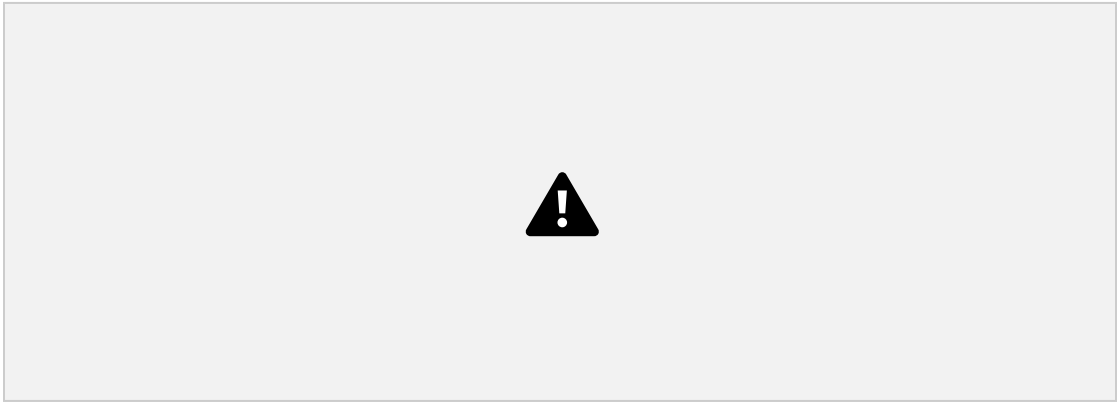
the others fields blank



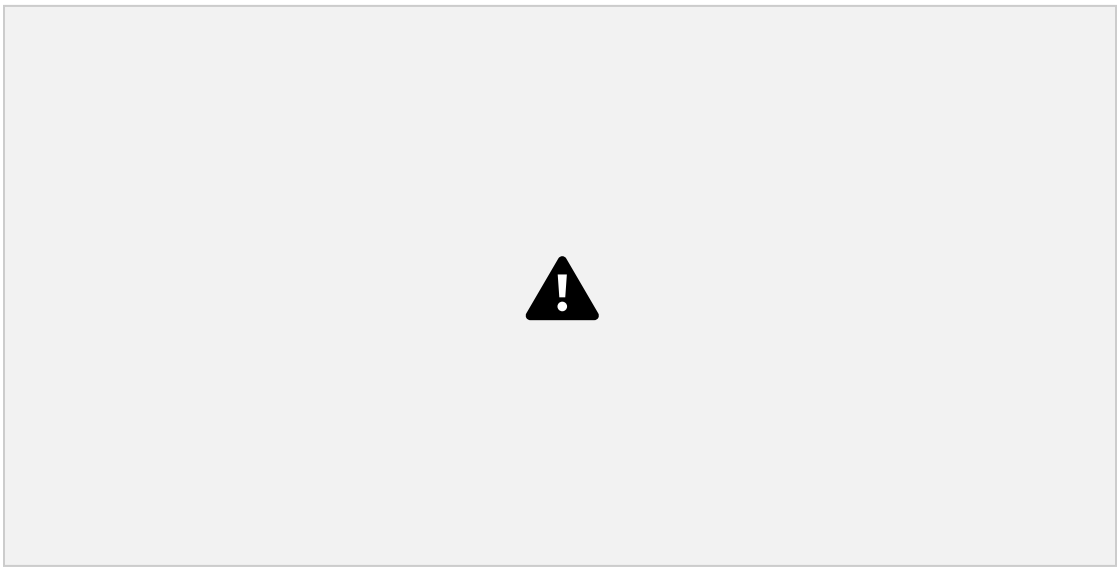
Add **Recording Controller**
Right click on the “Thread Group” and add a recording controller:**Add**=>**LogicController**=>**Recording Controller**



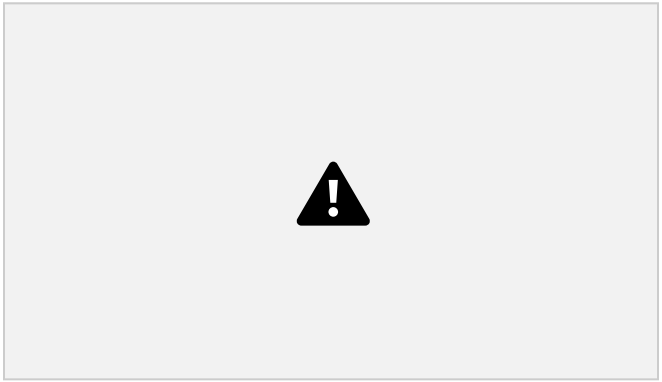
Add **HTTP(S) Test Script Recorder**
Right click on the Test plan and add the https test script recorder: **Add** => **Non-Test Elements** => **HTTP(S) Test Script Recorder**

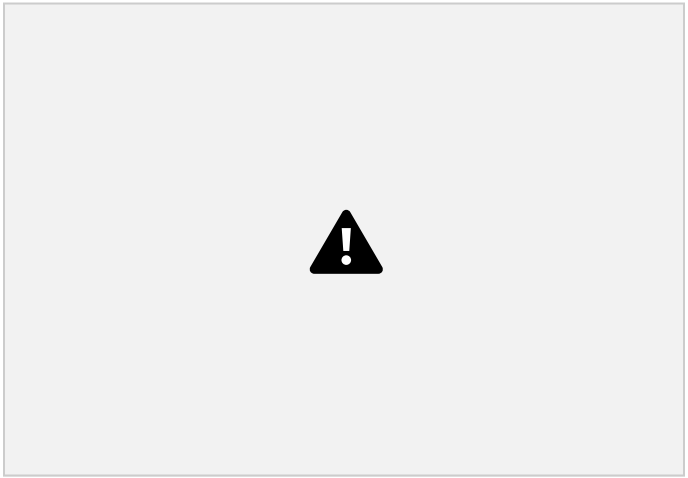


Set **Target Controller** where your recorded scripts will be added

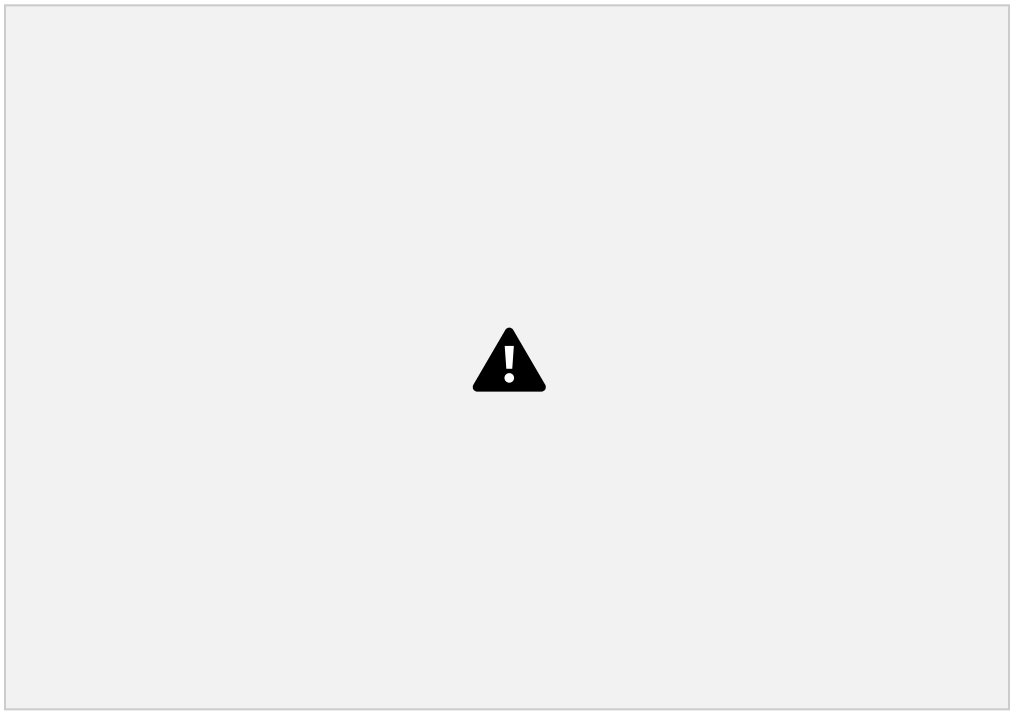


Click the **Start**. Now your JMeter proxy server start, The jmeter will issue you a CA certificate that will then be installed on your apache jmeter bin folder

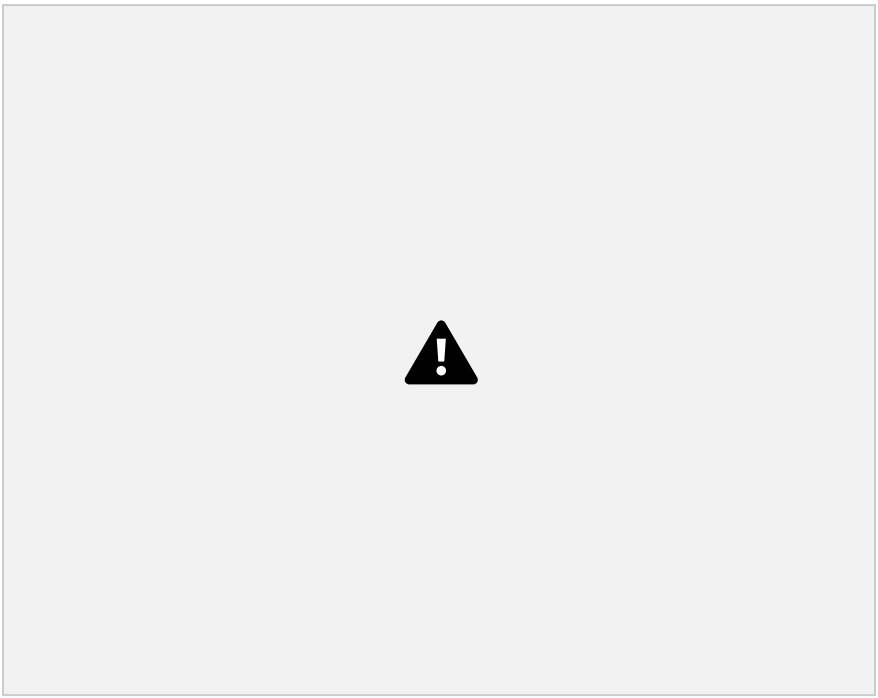




Now choose your browser I chose FireFox, go to settings - > certificate manager and import your Jmeter CA certificate here

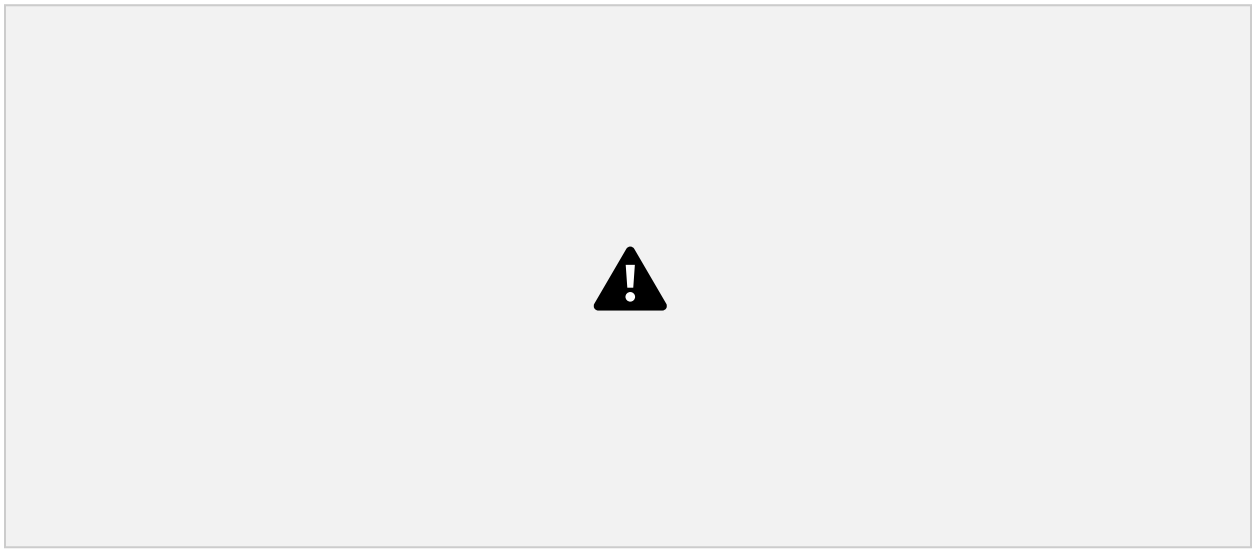


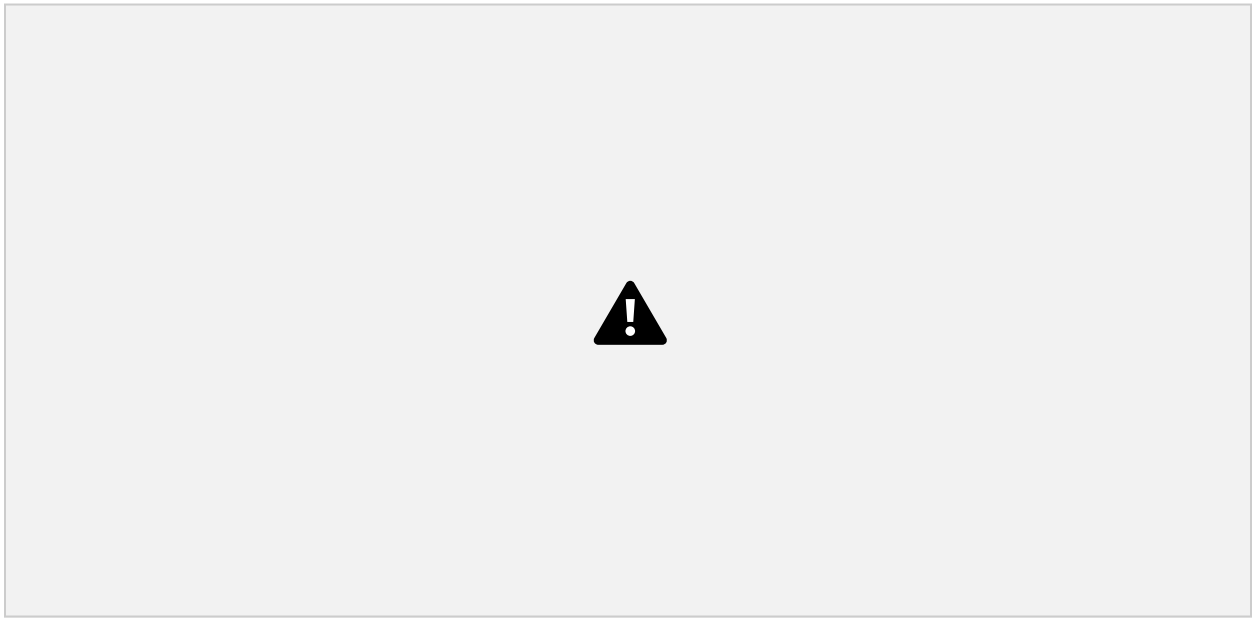
Now go to FireFox’s Network Setting and set them as following,

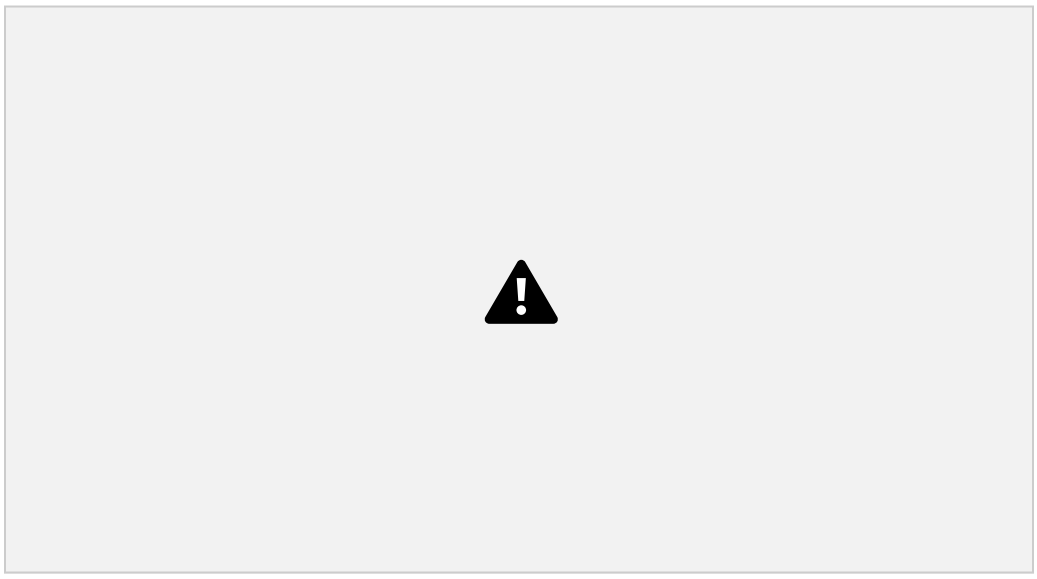


Step2: Record the activity:

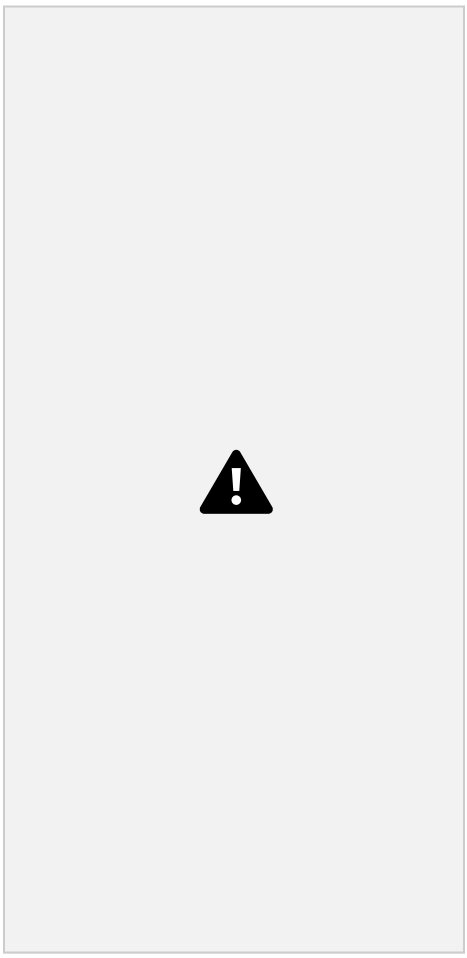
We launched google and carried out our browser activity.



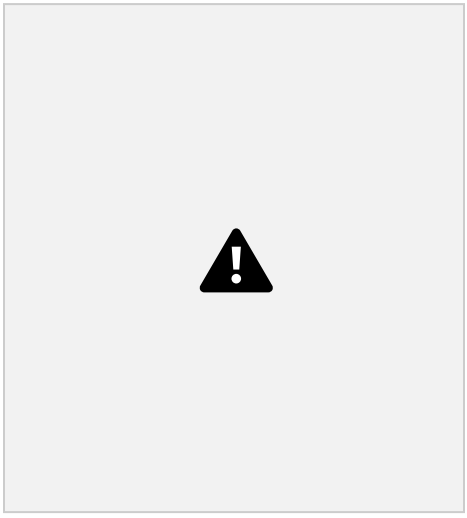


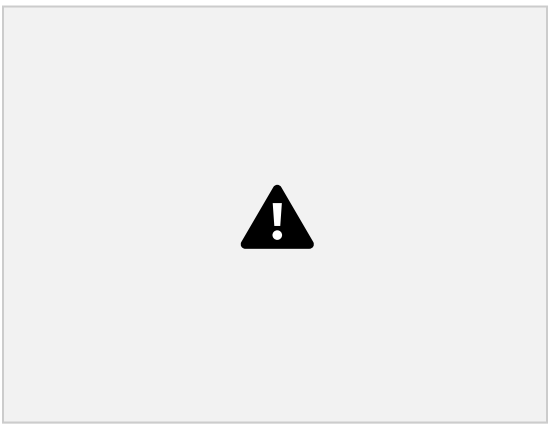


After finishing recording, you will see JMeter automatically created a new HTTP request as the figure below



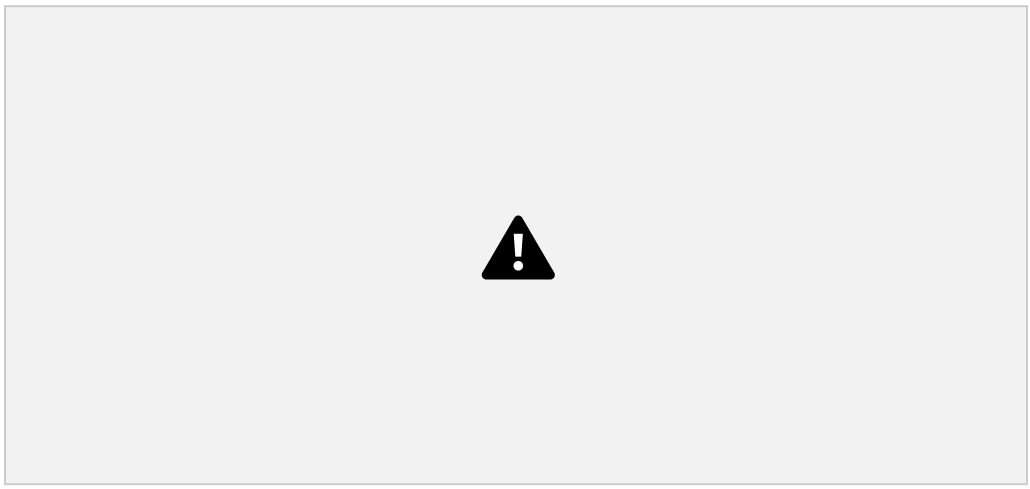
Click File => Save your Test Plan as





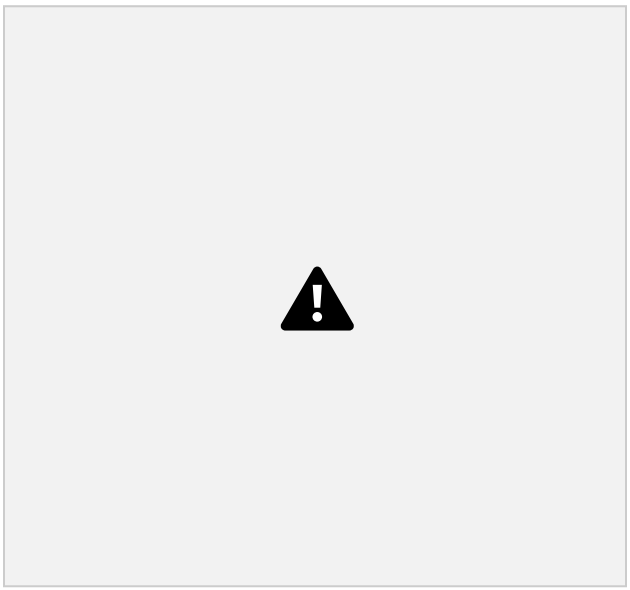
Step 3: Run your test plan

Select Thread Group => Add => Listener=> Summary Report

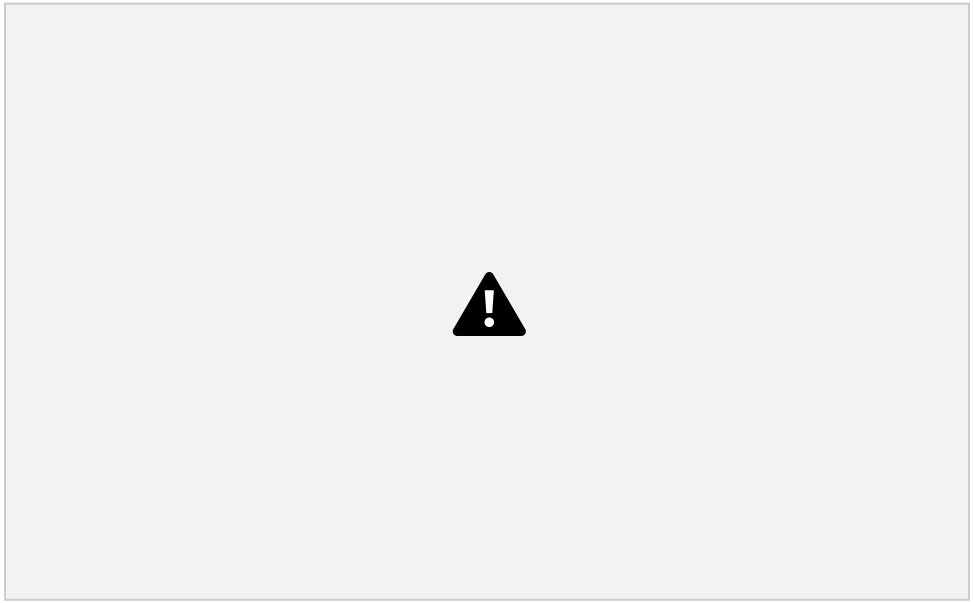


Summary Report Statistics:

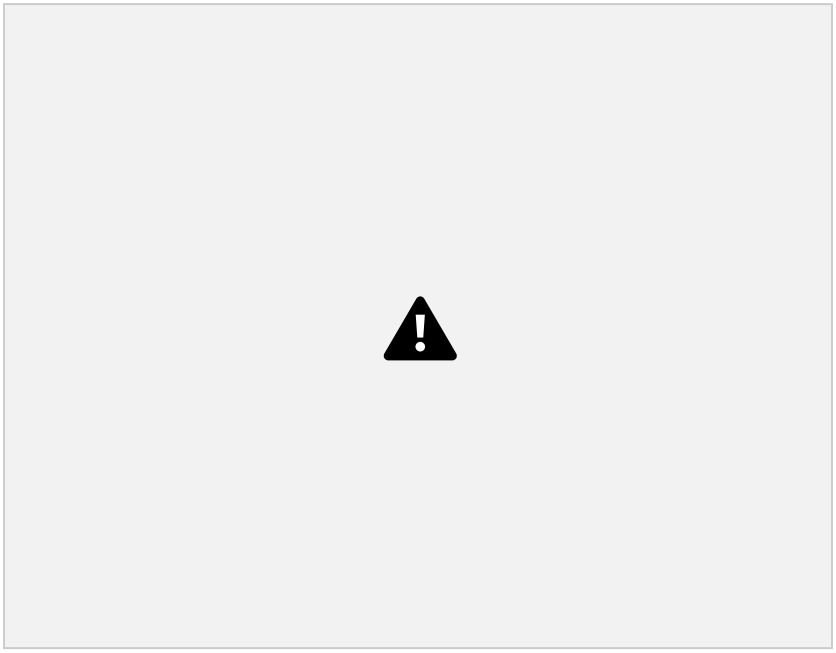
Label: See all the recorded HTTP request
Samples: Number of HTTP requests
Average: Average Response time for particular HTTP request
Min, Max: Minimum and maximum time taken by http request
Std Deviation: How many exceptional cases were found
Error: Denotes the error percentage in sample request
Throughput: Number of requests per unit time
Modify the number of threads in thread group to 100



Before you start the test, select “Summary Report”. When you ready to run a test, select Run => Start (Ctrl+R). JMeter will playback your activity in 100 times
As the test runs, the statistics will change until the test is done.



Save your test result by clicking on Save Table Data below,



What are Timers?

By default, JMeter sends the request **without pausing** between each request. In that case, JMeter could overwhelm your test server by making too many requests in a short amount of time.

Purpose of Timers:

Timers allow JMeter to delay between each request which a thread makes. A timer can solve the server overload problem.
Also, in real life visitors do not arrive at a website all at the same time, but at different time intervals. So Timer will help mimic the real-time behavior.

Following are some common types of a timer in JMeter

Constant Timer

Constant timer delays each user request for the same amount of time.

Gaussian Random Timer

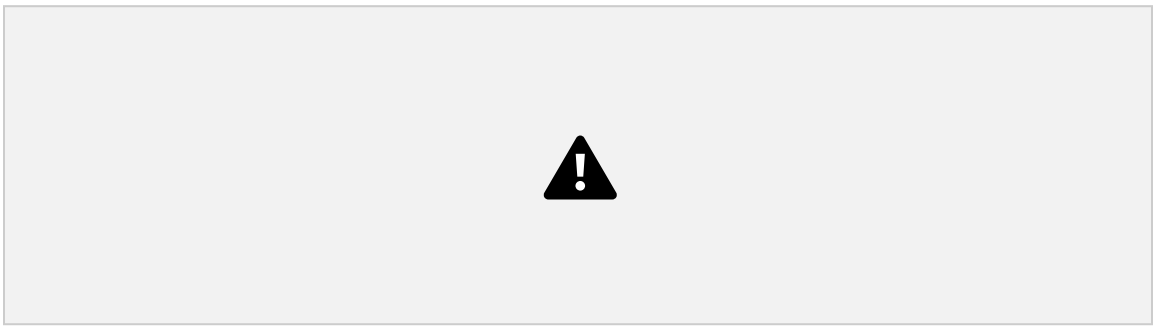
Gaussian random timer delays each user request for a random amount of time.

How to Use Constant Timer

In this example, you will use Constant Timer to set a fixed delay between user requests to google.com.
Let start with a simple test script

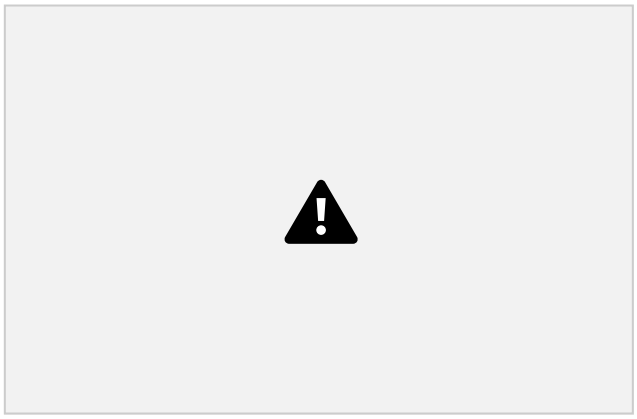
1. JMeter creates one user request to http://www.google.com 100 times
2. Delay between each user request is 5000 ms

Here is the roadmap for this practical example:



Step 1) Add Thread Group

Right click on the Test Plan and add a new thread group: Add-> Threads (Users) ->Thread Group

In Thread Group control panel, enter Thread Properties as following 

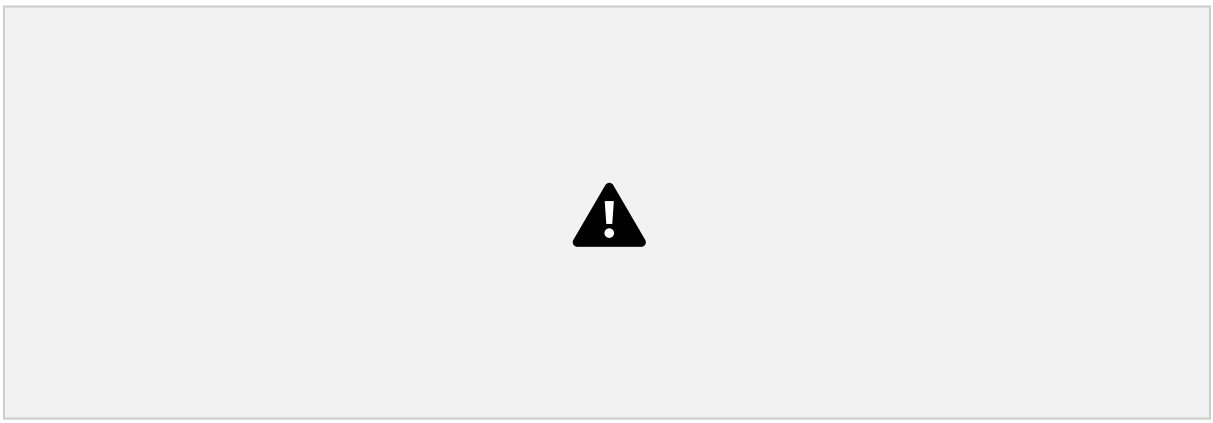
This setting lets JMeter create one user request to http://www.google.com in 100 times

Step 2) Add JMeter elements

- Add HTTP request default
- Add HTTP request

Step 3) Add Constant Timer

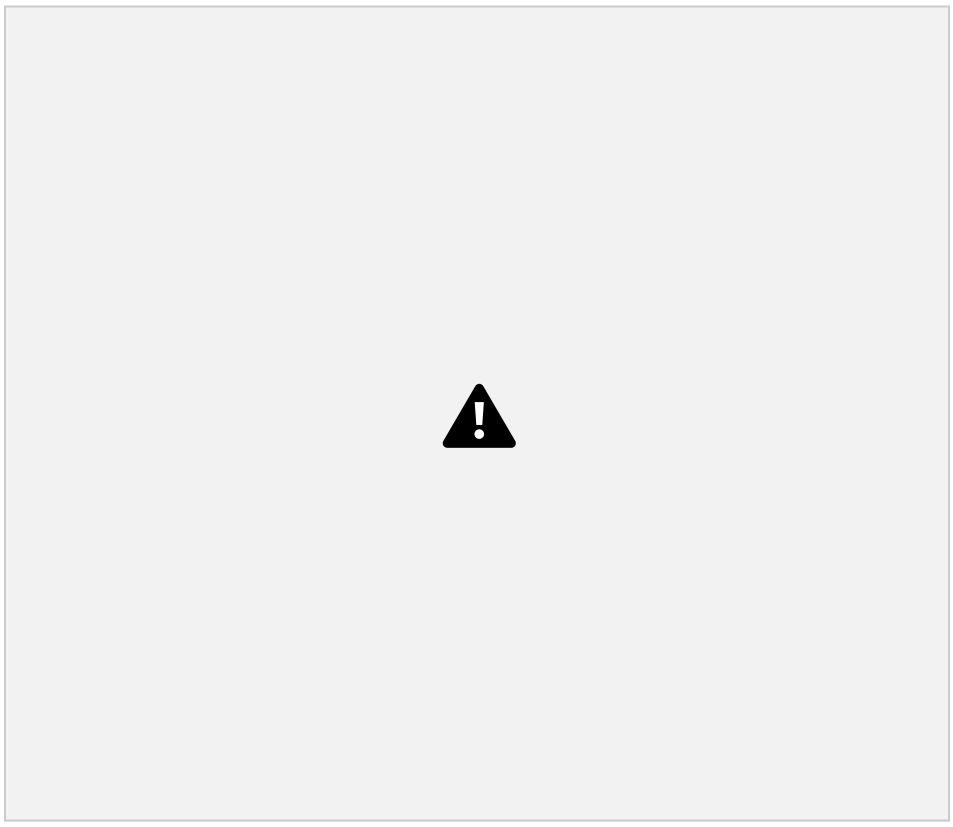
Right-click Thread Group -> Timer -> Constant Timer



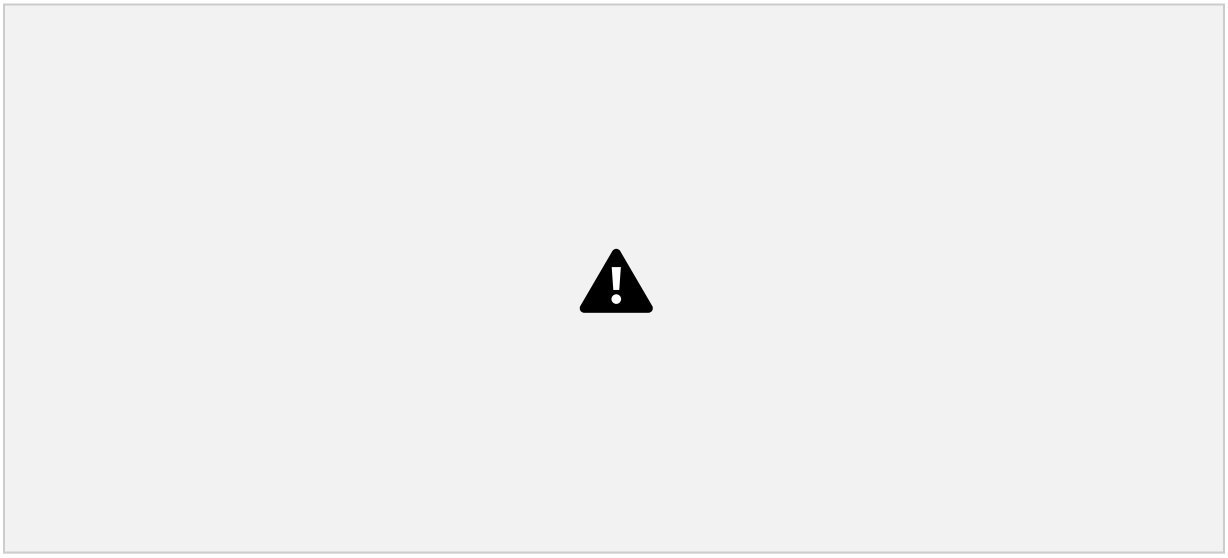
Configuring Thread Delay of 5000 milliseconds

Step 4) Add View Results in Table

View Results in Table displays the test result in table format.
Right click Add -> Listener ->View Result in Table

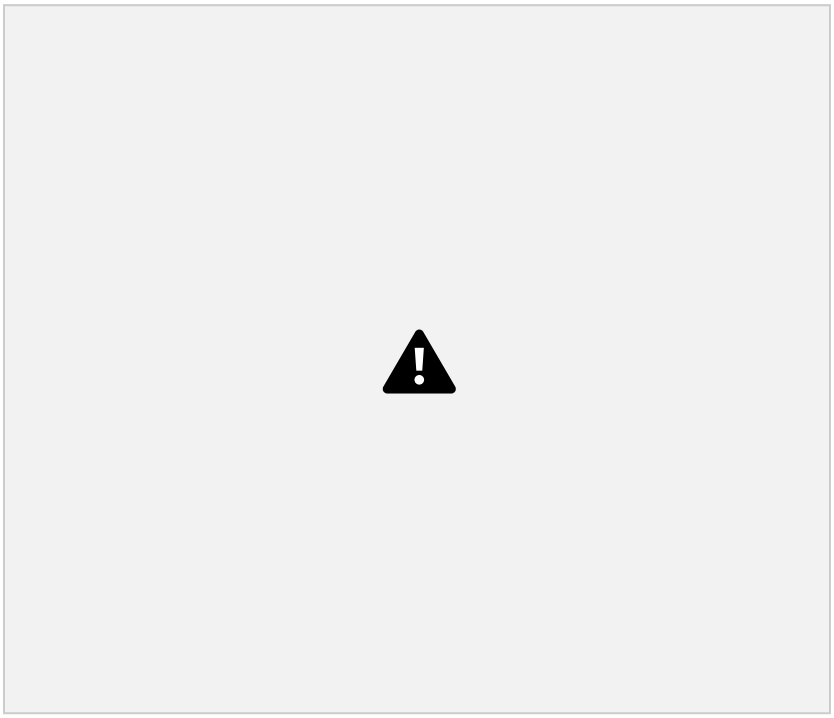


View Results in Table displays as below figure



Step 5) Run your test

When you ready to run a test, click the Run button on the menu bar, or short key Ctrl+R
This is the result of this test



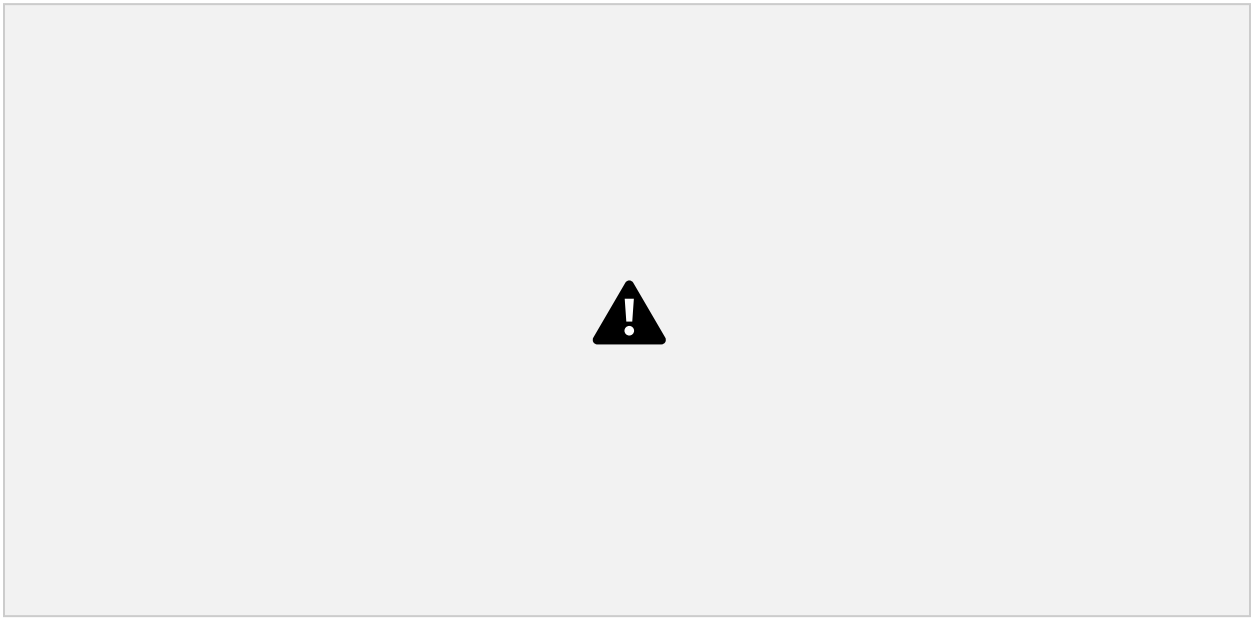
For example, in the above figure, let analyze the Sample 2

- Start time is 22:05:01.866
- Sample Time of Sample 2 is 172 ms
- Constant Timer: 5000 ms (as configured)
- End Time of this sample is = 22:05:01.866 + 172 + 5000 = 22:05:07.038

So the Sample 3 should start at the time is 22:05:07.039 (As shown in the above figure)

The delay of each sample is 5000 ms

If you change the Constant Timer is zero, you will see the result is changed



Let analyze the Sample 1

- Start time is 22:17:39.141
- Sample Time of Sample 2 is 370 ms
- Constant Timer : 0 ms (as configured)
- End Time of this sample is = 22:17:39.141+ 370 + 0 = 22:17:39.511

So the Sample 2 should start at the time is 22:17:39.512

What is an Assertion?

Assertion help verifies that your server under test returns the expected results.

Types of Assertions

Following are some commonly used Assertion in JMeter: ● Response Assertion

- Duration Assertion
- Size Assertion
- XML Assertion
- HTML Assertion
- Steps to use Response Assertion

Response Assertion

The response assertion lets you add pattern strings to be compared against various fields of the server response.
For example, you send a user request to the website http://www.google.com and get the server response. You can use Response Assertion to verify if the server response **contains** expected pattern string (e.g. “OK”).

Duration Assertion

The Duration Assertion tests that each server response was received within a **given amount** of time. Any response that takes longer than the given number of milliseconds (specified by the user) is marked as a failed response.
For example, a user request is sent to www.google.com by JMeter and get a response within **expected** time 5 ms then Test Case pass, else, test case failed.

Size Assertion

The Size Assertion tests that each server response contains the expected number of bytes in it. You can specify that the size be equal to, greater than, less than, or not equal to a given number of bytes.
JMeter sends a user request to www.google.com and gets response

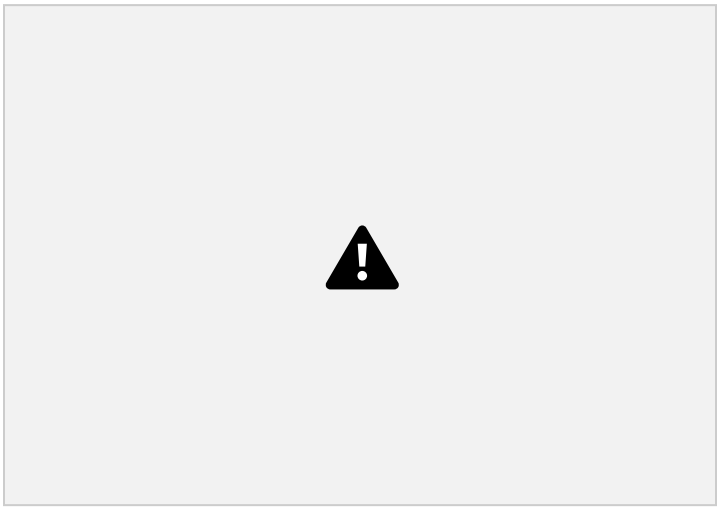
packet with size less than **expected** byte 5000 bytes a test case pass. If else, test case failed.

XML Assertion

The XML Assertion tests that the response data consists of a formally correct XML document.

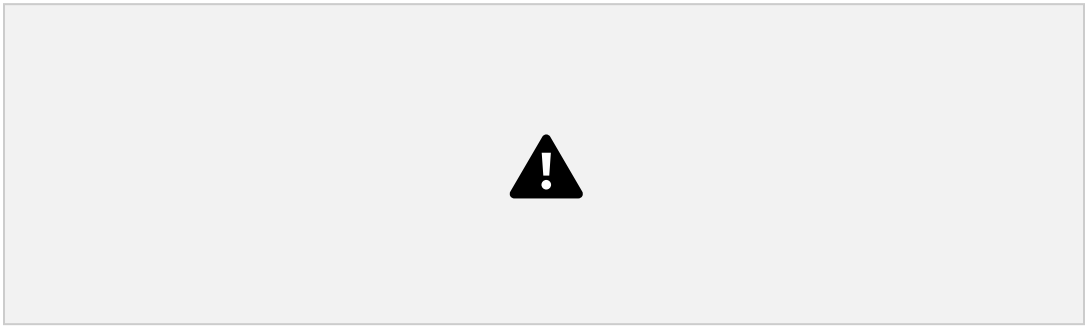
HTML Assertion

The HTML Assertion allows the user to check the HTML syntax of the response data. It means the response data must be met the HTML syntax.



Steps to use Response Assertion

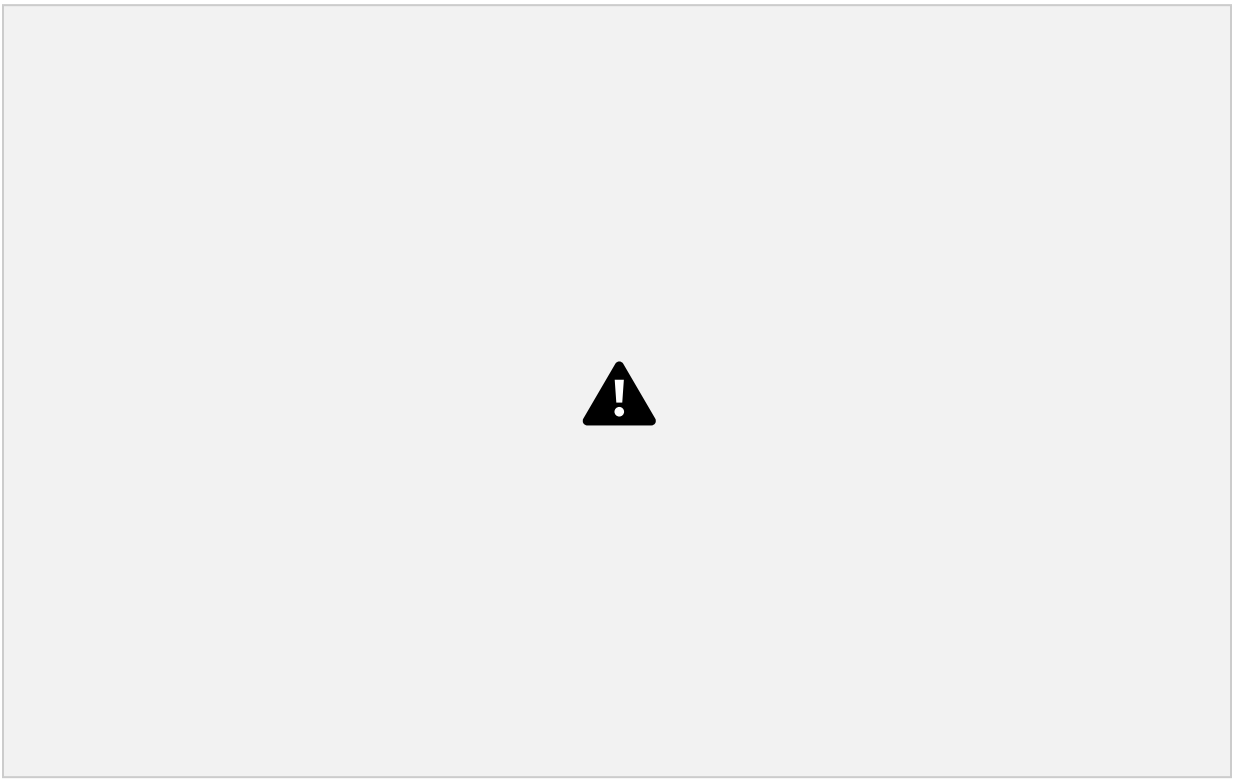
We will continue on the script we developed in the earlier tutorial. In this test, we are using Response Assertion to compare the response packet from www.google.com matches your expected string. **Here is the roadmap for this test:**



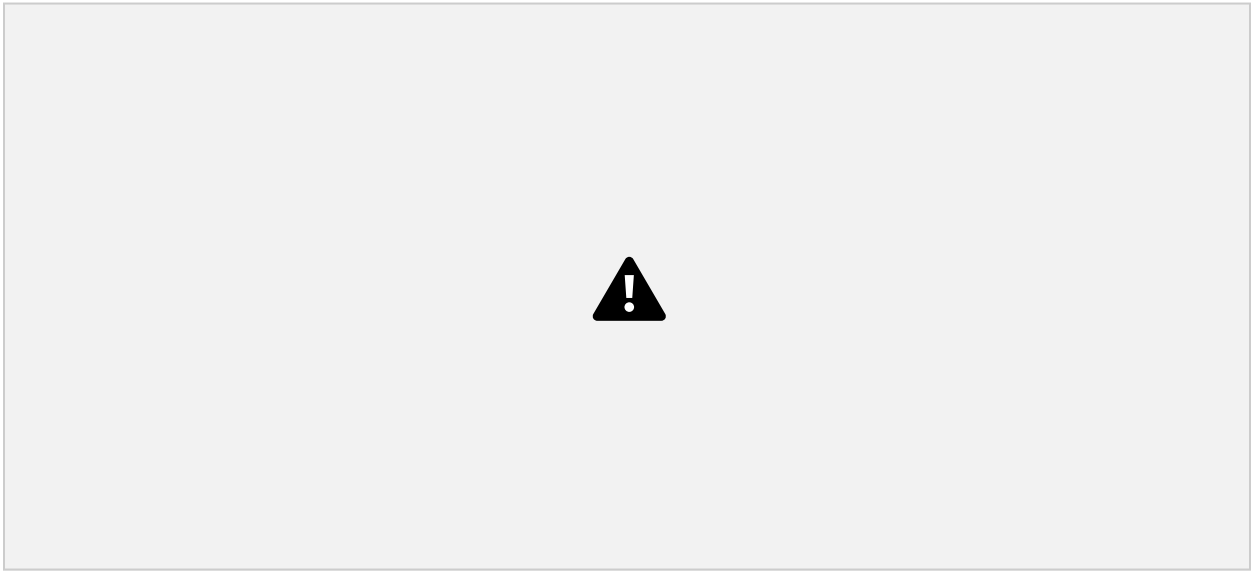
The response assertion control panel lets you add pattern strings to be compared against various fields of the response.

Step 1) Add Response Assertion

Right-Click Thread Group -> Add -> Assertions -> Response Assertion



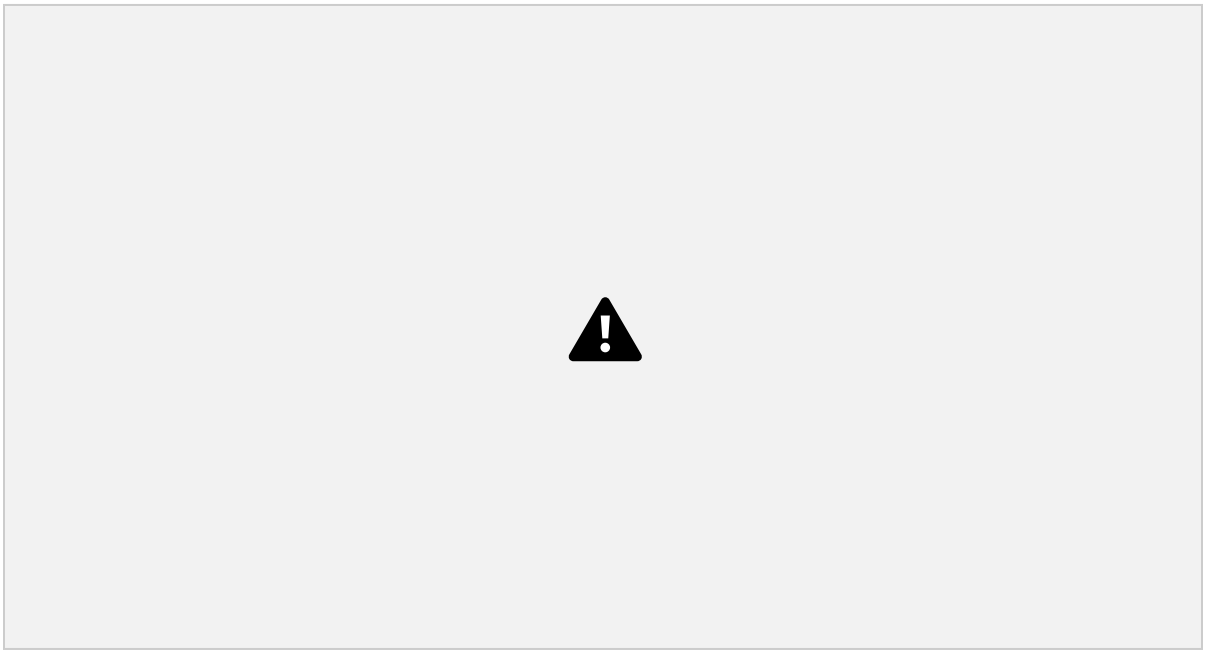
Response Assertion Pane displays as below figure:



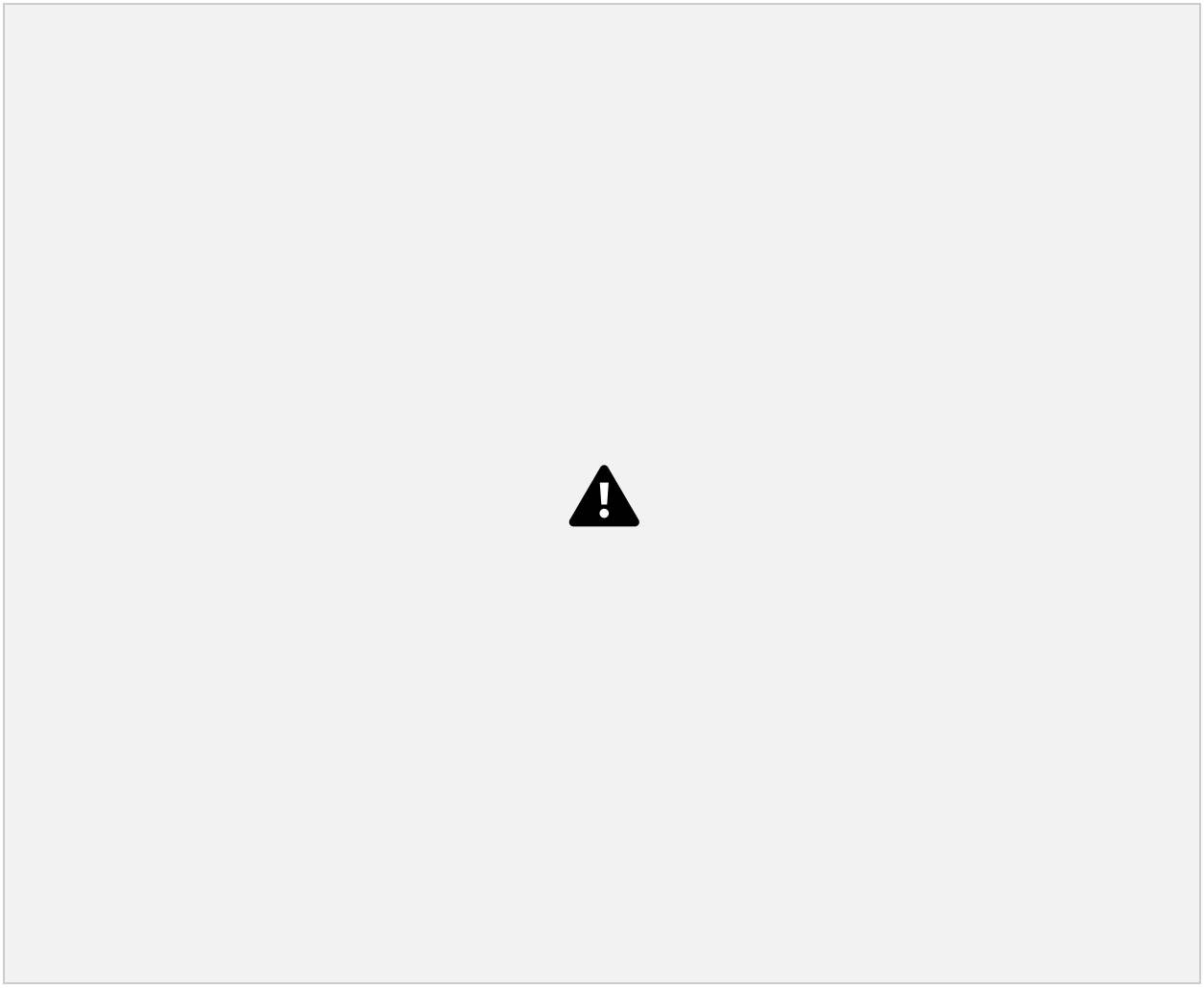
Step 2) Add Pattern to test

When you send a request to Google server, it may return some response code as below:

- **404: Server error**
- **200: Server OK**
- **302:** Web server redirects to other pages. This usually happens when you access google.com from the outside USA. Google redirects to country-specific website. As shown below, google.com redirects to google.co.in for Indian Users.

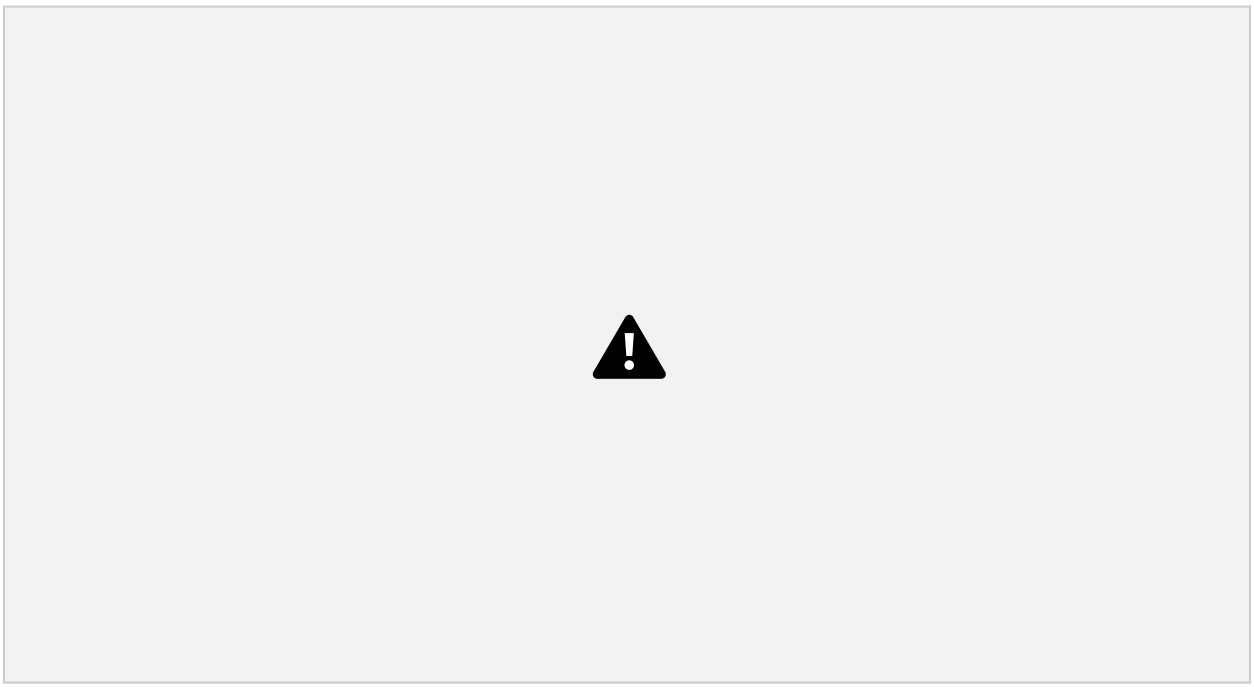


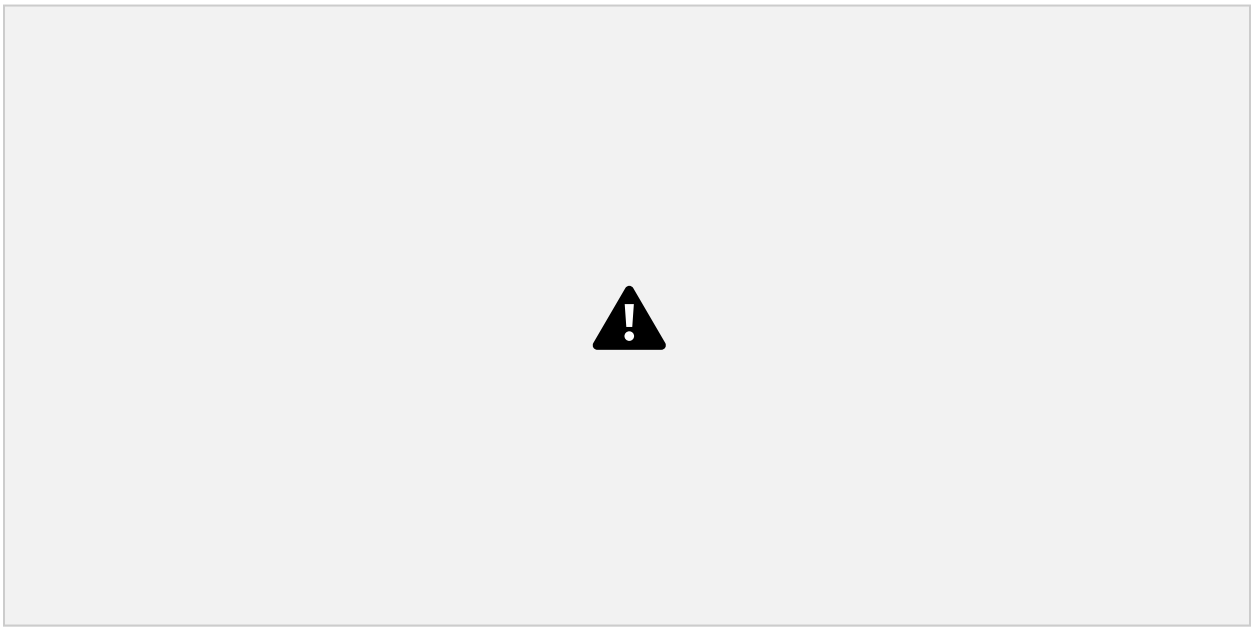
Assume that you want to verify that the web server google.com responses code contains pattern 302,
On Response Field To Test, choose Response Code, On Response Assertion Panel, click Add -> a new blank entry display -> enter 302 in Pattern to Test.



Step 3) Add Assertion Results

Right click Thread Group, Add -> Listener -> Assertion Results

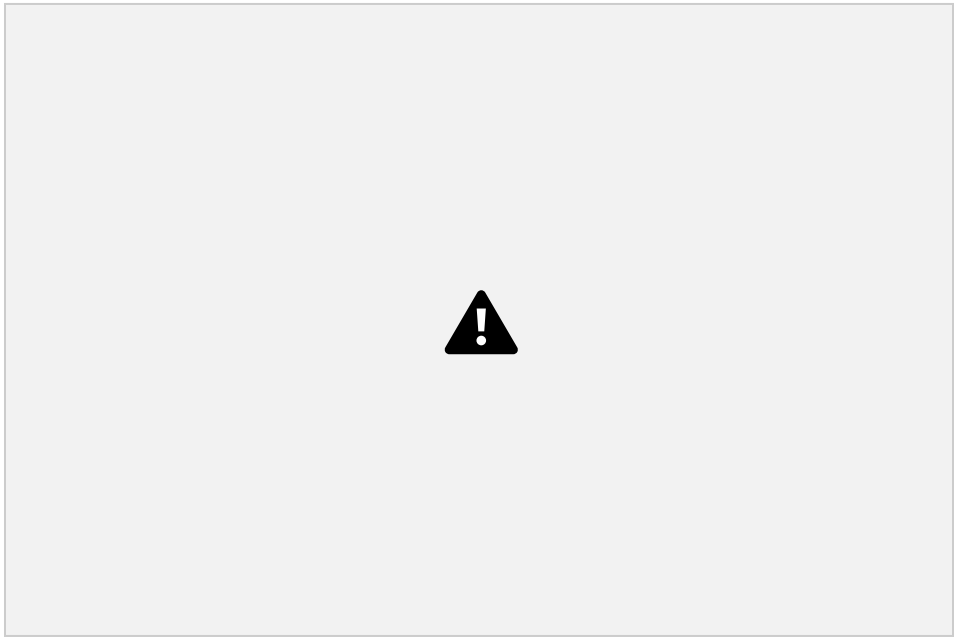




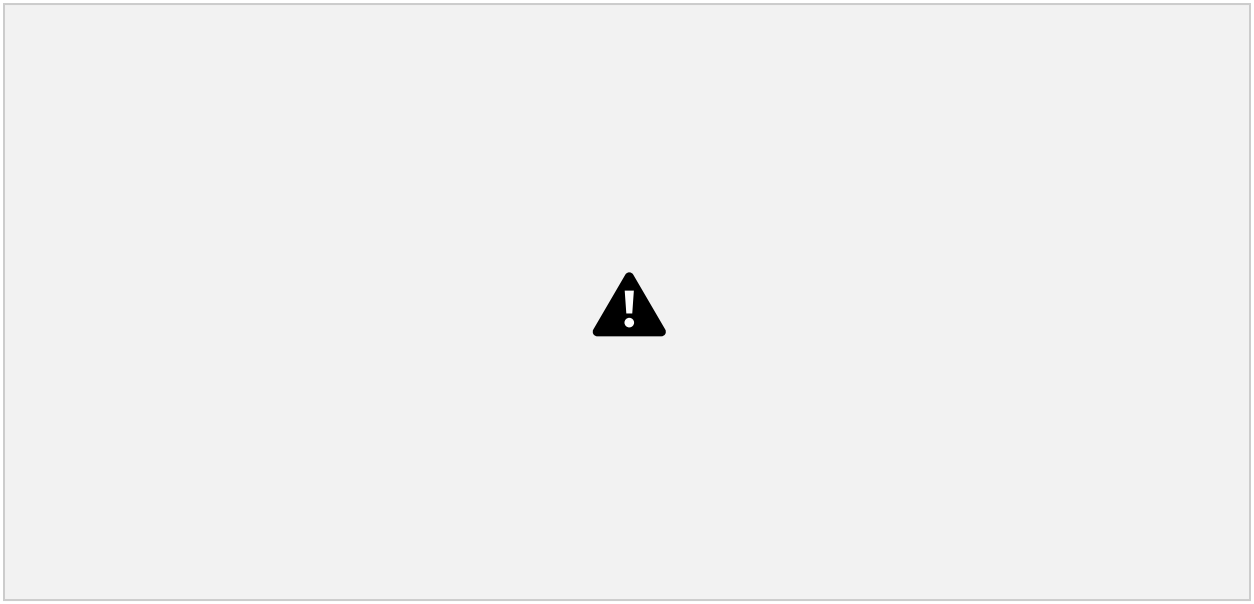
Step 4) Run your test

Click on Thread Group -> Assertion Result
When you ready to run a test, click the Run button on the menu bar, or short key Ctrl+R.

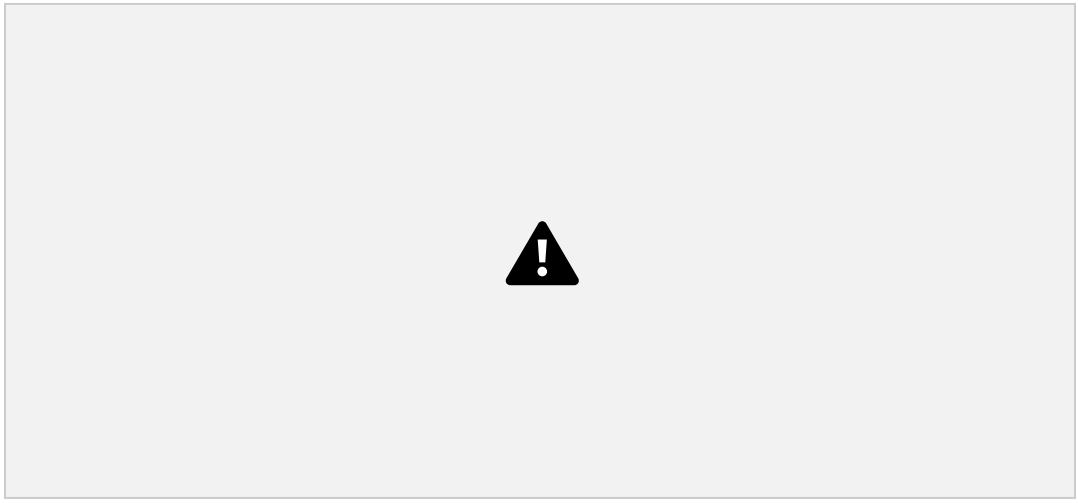
The test result will display on the Assertion Results pane. If Google server response code contains the pattern 302, the test case is passed. You will see the message displayed as follows:



Now back to the Response Assertion Panel, you change the Pattern to test to from 302 to 500.



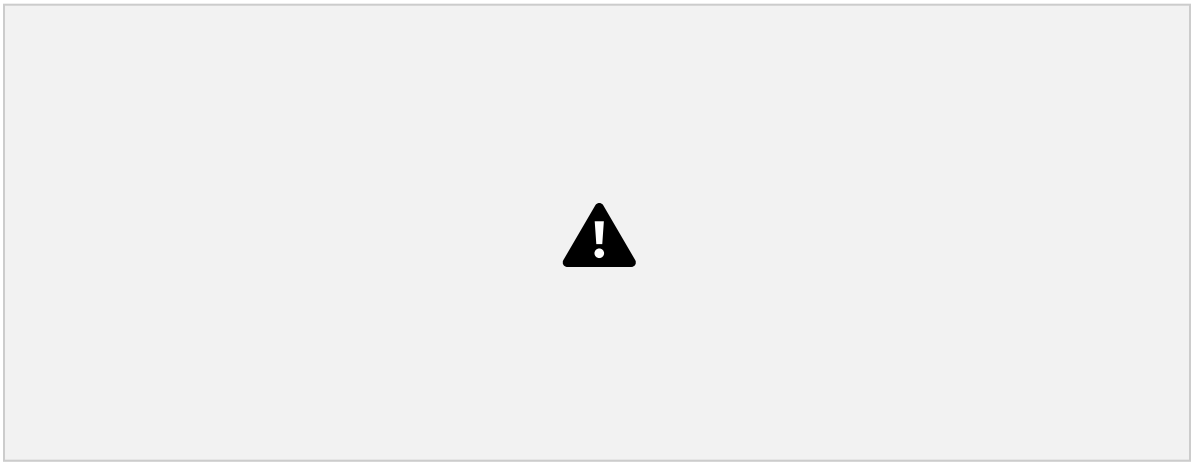
Because Google server response code doesn’t contain this pattern, you will see the test case Failed as following:

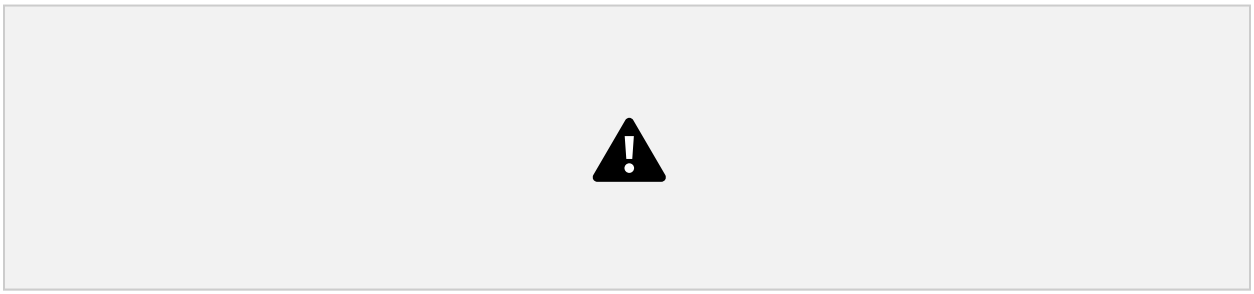


In **JMeter**, a **Controller** is used to control the flow of execution within a test plan. Controllers define the order in which requests (or samplers) are executed and how they behave. There are two main types of controllers in JMeter: Simple Controllers and Logic Controllers.

1. Simple Controller

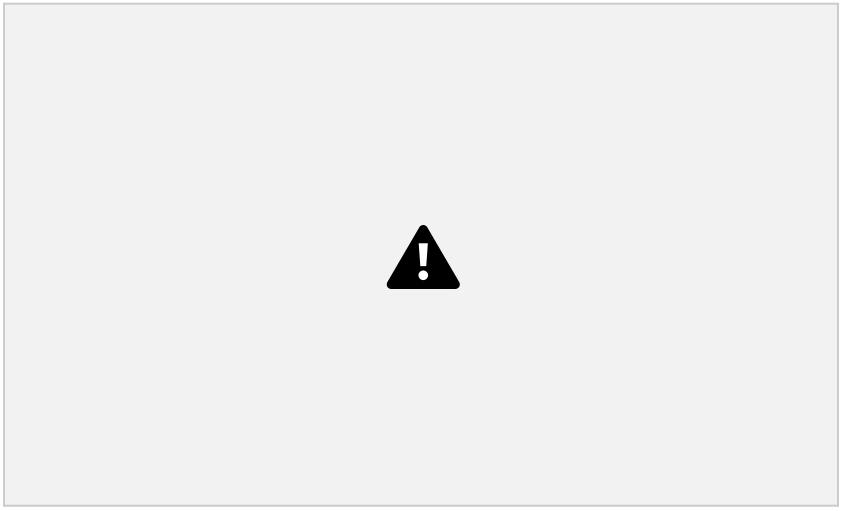
● **Purpose**: Organizes samplers and other elements within a test plan. ● **Behavior**: Does not control execution flow; it simply groups elements together. ● **Use Case**: Used for organizing and making test plans more readable.

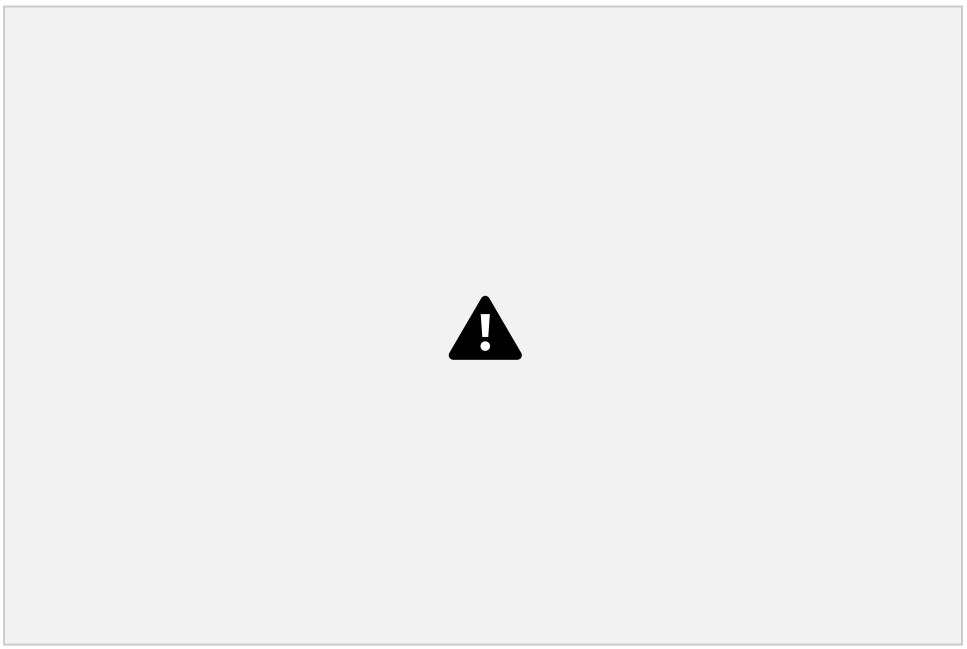




2. Logic Controllers

- **Purpose**: Controls the flow of execution based on conditions or loops.
- **Use Case**: Defines how and when samplers are executed.



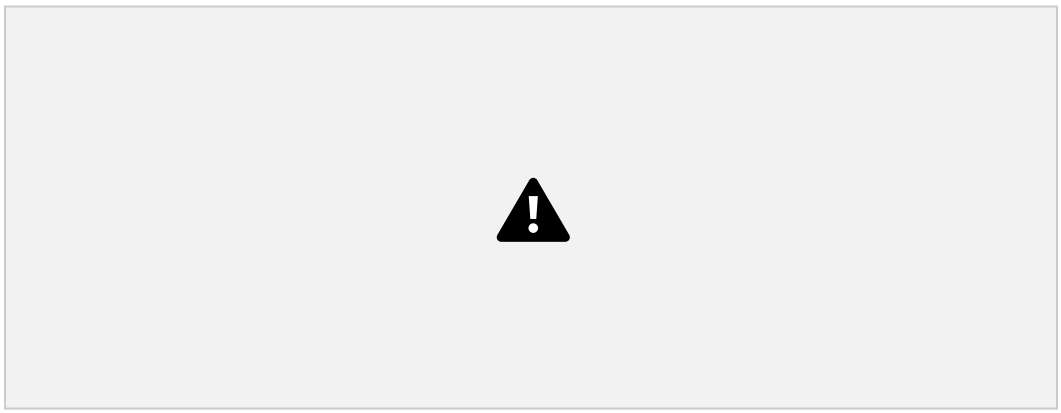


Common Logic Controllers:

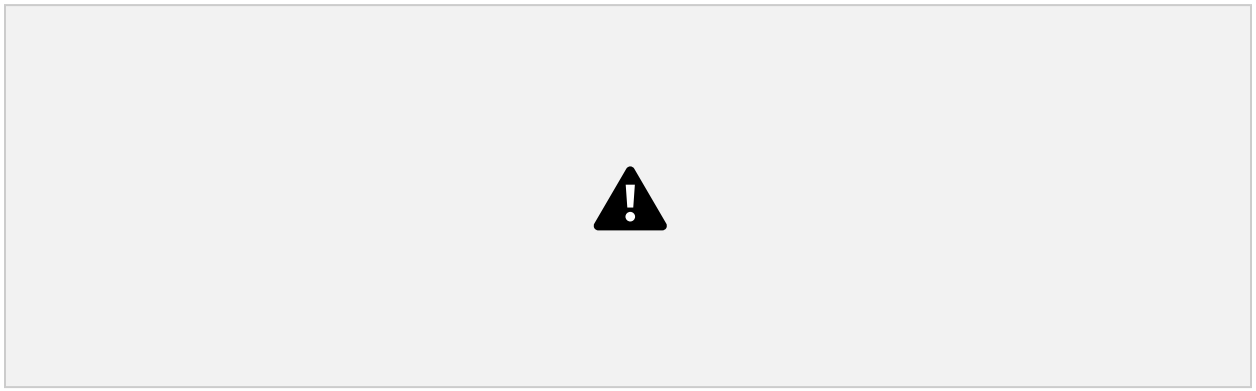
- **Thread Group**:
    - Defines the number of threads (virtual users) and their behavior (e.g., how many times to loop).

- **If Controller**:
    - Executes samplers inside it based on a condition (similar to an "if" statement).

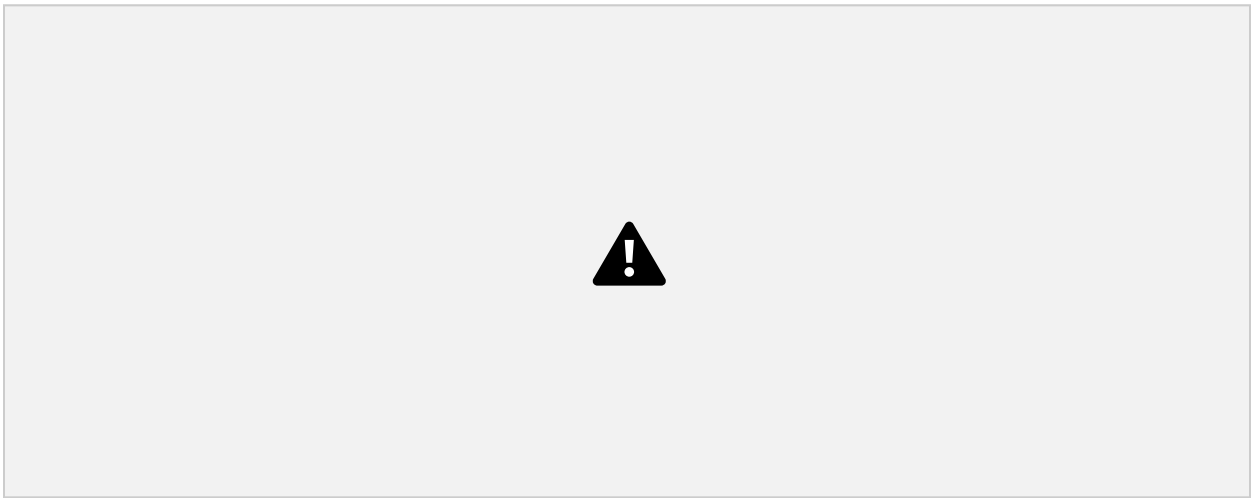
- **Loop Controller**:
    - Repeats samplers a specified number of times or indefinitely.



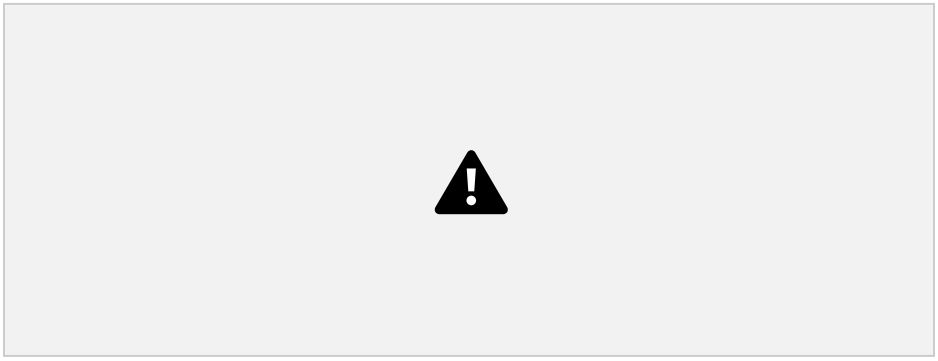
- **While Controller**:
    - Executes samplers as long as a given condition evaluates to true.



- **ForEach Controller**:
    - Loops through a collection of variables or data, executing samplers for each item in the collection.

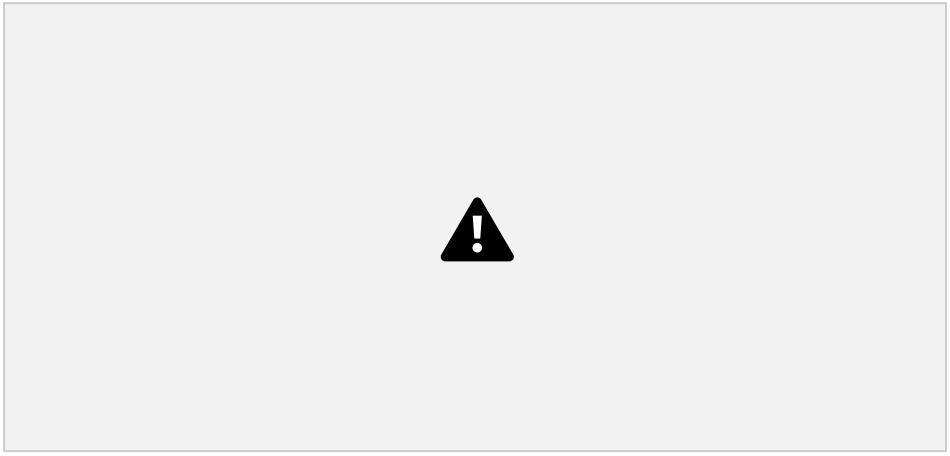


- **Throughput Controller**:



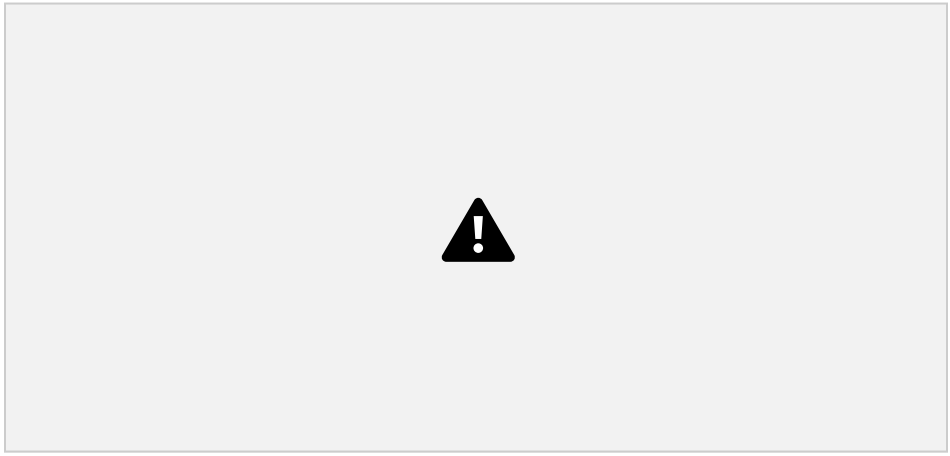
Pre-Processors in JMeter:

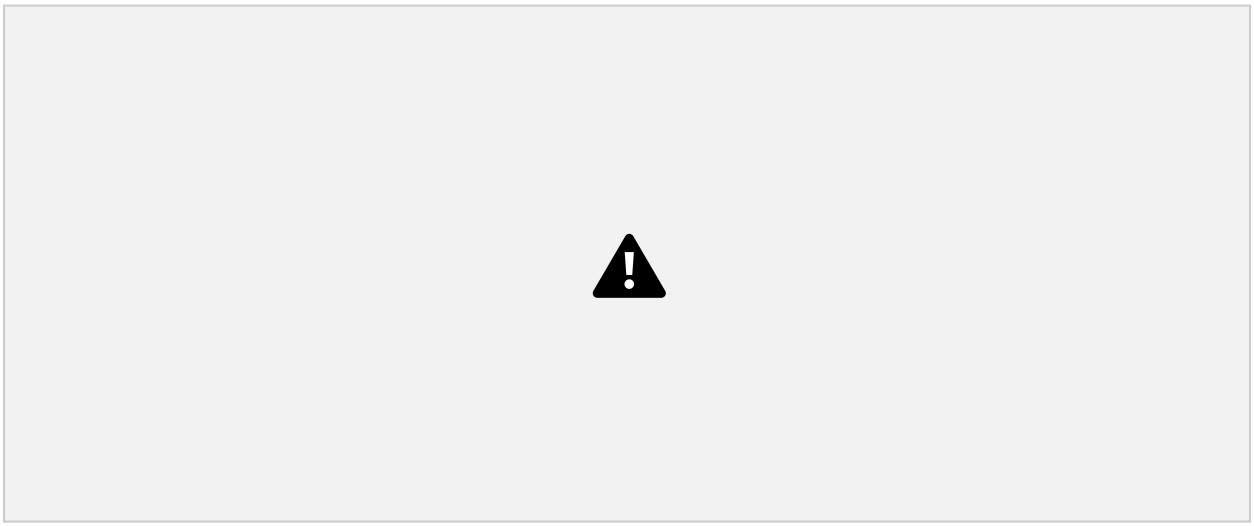
- **Executed before a request is sent.**
- **Purpose: Modify or prepare request data.**
- **Common Uses:**
    - **Setting dynamic parameters (e.g., session tokens).**
    - **Authenticating users or fetching data before requests.**
    - **Modifying HTTP headers or bodies.**



Post-Processors in JMeter:

- **Executed after a request is sent and response is received.**
- **Purpose: Extract or process data from the response.**
- **Common Uses:**
    - **Extracting values (e.g., session IDs, tokens) from responses using regex, JSON, or XPath.**
    - **Storing extracted data for use in subsequent requests.**
    - **Handling dynamic content like API tokens.**





The End